

Laboratory Journal for

Software Design Lab

(ITL601)

Semester VI

Bachelor's Degree in Information Technology



Don Bosco Institute of Technology, Mumbai 400070
(Affiliated to the University of Mumbai)
Department of Information Technology

Academic Year 2020-2021

INDEX

Subject: Software Design Lab

Faculty: Ms. Vaishali Kavathekar

S. No.	Topic	Page no.	Performance date	Submission date
1	To formulate the problem statement and scope of the mini project	1	29/3/21	10/4/21
2	To prepare the software requirement specification (SRS) document	6	29/3/21	10/4/21
3	Project Cost Estimation	17	29/3/21	10/4/21
4	Structural Modeling (Object, Class , component, deployment)	25	12/4/21	30/4/21
5	Behavioral Modeling-1 (Use- Case , Sequence and Collaboration Diagram)	31	12/4/21	30/4/21
6	Behavioral Modeling-2 (Activity & State Chart Diagram)	33	12/4/21	30/4/21
7	Software Design Document	35	26/4/21	24/5/21
8	WBS	48	26/4/21	25/5/21
9	Schedule Management Plan with network diagrams and time-line charts.	52	26/4/21	20/5/21

Names & Roll Nos. of Group Members:	{	1	<u>Amish Thekke Parambil (Roll.no 59)</u>
		2	<u>Abhijeet Ringe (Roll no. 47)</u>
		3	<u>Kshitij Rao (Roll no. 45)</u>
Mini Project Topic Name:		<u>Smart Door Lock System</u>	
Date:		<u>25/5/2021</u>	

This is to certify that the above students have successfully completed the Term work in the subject of **Software Design Lab** during **TE Semester VI** in the Academic Year **2019 – 2020**.

Vaishali Kavathekar
Faculty In charge

DON BOSCO INSTITUTE OF TECHNOLOGY
Department of Information Technology

T.E IT Sem VI

2021

Experiment No:1

Title: To Formulate the Problem statement and scope

Pre requisites: Background of Software Engineering (The Need)

Theory:

Hardware & software requirements

Python Modules used:

- OpenCV
- TF-IDF Vectorizer
- Numpy
- Django
- LBPH Classifier

Hardware Components:

- Raspberry Pi 4-2GB RAM
- 2 Channel Relay
- Pi Camera 5MP
- Breadboard Half Size
- Electronic Door Strike
- 12V Power Supply
- ISD1820 Voice Recording Module With On Board Mic and Speaker

Procedure:

1. Form a group of 3-4(max) members to work on the mini project.

Group Members:

Kshitij Rao - 45

Amish Thekke Parambil - 59

Abhijeet Ringe - 47

2. Do literature survey to identify the topic for mini project.

To design such a product, we conducted a thorough research to see the products currently present in market. Main focus was to provide more security and features in the system.

- Over the course of last decade, home security and automation systems have been gaining extreme popularity because of the enhancement in the field of science and the concomitant risks of breaking in the system.
- Most door systems are used for streaming video to look who's there outside, at the door. None of

them provide an automatic lock/unlock system and tracks the number of people, who came to house/office.

- Also most of the lock solutions, don't provide a voice assistant to guide people, who are not a member/ employee of the house/office.
- As per a technical survey conducted by CORUM Security, it shows that Facial recognition locks are more secure, compared to other traditional and modern lock systems.

3. Finalise the project topic after discussing the brief scope with the faculty in charge.

Finalised Project Topic - "SMART DOOR LOCK SYSTEM"

4. Formulate the problem statement.

To increase security of the current lock systems along with keeping a record of people and storing messages left by people. It should serve the purpose of remaining contactless, and also provide a suitable solution which is more secure, smart and is faster and efficient than current door lock systems.

5. Define the scope of the mini project in a tabular format as shown below:-

Application Features	Application of Convolutional Neural Networks in Face Recognition	Door Assistant chatbot using Natural Language Processing	Django Framework (RESTful API,AJAX)	Mini Project
	https://ieeexplore.ieee.org/document/8663009	https://www.researchgate.net/publication/340678278_A_Smart_Chatbot_Architecture_based_NLP_and_Machine_Learning_for_Health_Care_Assistance	https://www.researchgate.net/publication/254006913_Web_development_with_python_and_django_abstract_only	
Register and Recognize user through face detection.	Y	N	Y	Y
Conversational Chatbot to guide unregistered users.	N	Y	Y	Y
OTP System	N	N	Y	Y
Log Tracking of Registered People.	N	N	Y	Y

6. Identify the hardware–software requirements for the mini project.

Project Description:

- Smart Door Lock System with Facial Recognition and Voice Assistant
- System Interface hosted on Django Server and designed using HTML, CSS, Bootstrap 4 etc.
- Backend Database: SQLite

Website Functionalities:

- Live Video Streaming from the Lock System, displayed in the website
- Display Visitor Logs from Database Data
- Member Registration with Facial Data Capture for Training Facial Recognizer
- Sending unlock request over Internet, after OTP Verification
- Authorization Level Based Access to Interface
- To show the conversational workflow of the Voice Assistant in the Website

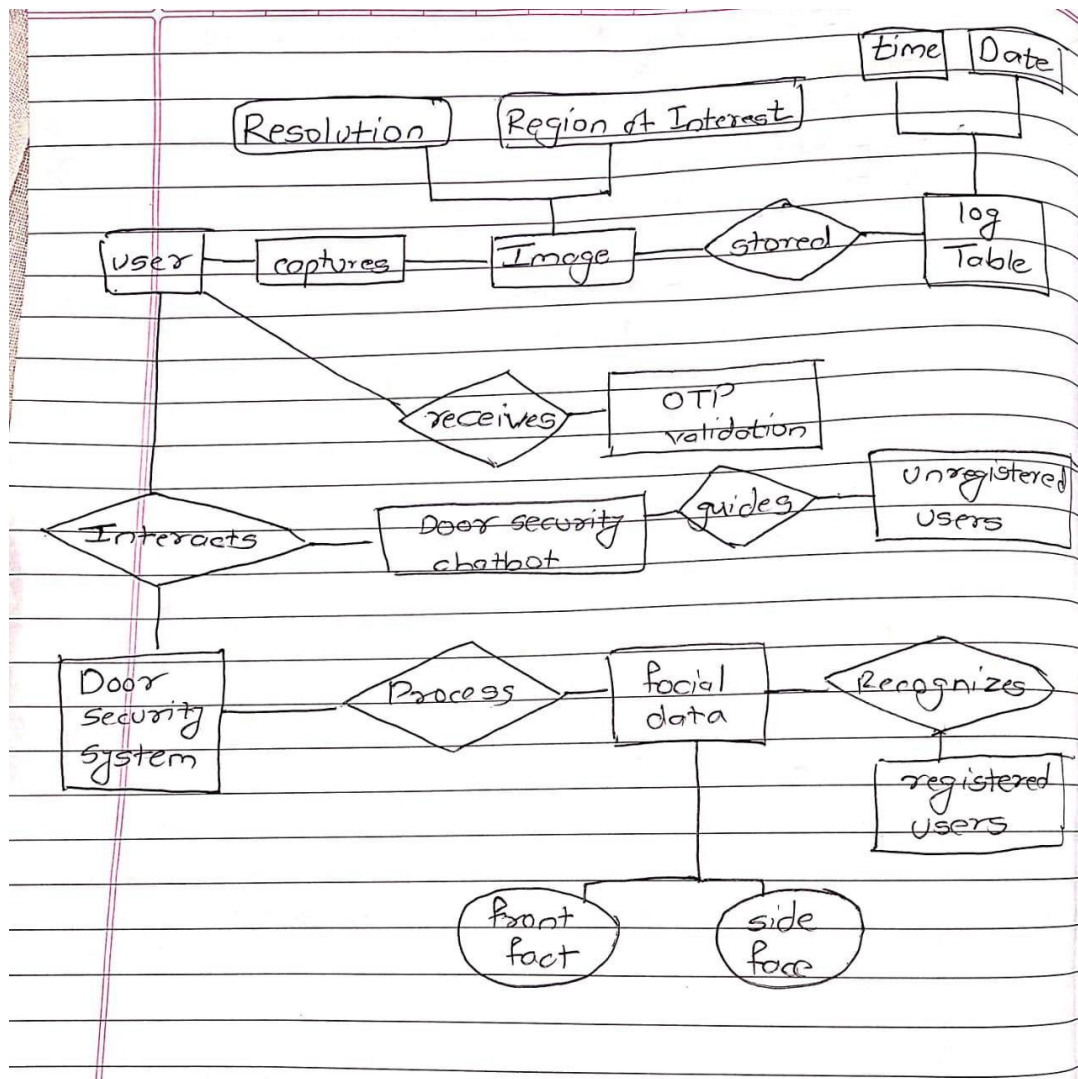
Python Modules used :

- OpenCV
- TF-IDF Vectorizer
- Numpy
- Django
- LBPH Classifier

Hardware Components

- Raspberry Pi 4-2GB RAM
- 2 Channel Relay
- Pi Camera 5MP
- Breadboard Half Size
- Electronic Door Strike
- 12V Power Supply
- ISD1820 Voice Recording Module With On Board Mic and Speaker

7. Draw the E-R diagram for the project.



Conclusion:

As per our research, there's no lock system in Indian markets, which provide both facial recognition as well as a voice assistant for guidance to delivery people and guests. We were able to design a simulated process of:

- System Interface in a Web App
- How the logs are saved and displayed in the Web App
- Conversational workflow of the voice assistant
- User Registration and Identification using Facial Recognition
- Video Streaming viewable on the Web App

We can easily build this project and can also market this as solution for smart home security. Such a smart lock system will be accessible to all people and can be easily added to homes, to give a sense of high level of modern type security to people.

References:

- [1] HTML5 and CSS W3 Schools <https://www.w3schools.com/html/default.asp>
- [2] Bootstrap4 <https://getbootstrap.com/docs/4.0/getting-started/introduction/>
- [3] Web Service API: 2Factor API <https://2fa.api-docs.io/v1/getting-started-with-2factor-api>
- [4] Web Server and Security Django (<https://docs.djangoproject.com/en/3.1/>) helped in understanding integration of Python Code and how to execute queries to store/extract data in SQLite Databases in the Web App.
- [5] Facial Recognition and Facial Data Capturing LBPH Classifier (https://docs.opencv.org/3.4/df/d25/classcv_1_1face_1_1LBPHFaceRecognizer.html) was used to train facial data, for recognition system of the Door Lock.
- [6] Raspberry Pi Camera Module (<https://www.raspberrypi.org/products/raspberry-pi-4-model-b/>)
- [7] Raspberry Pi 4 (<https://picamera.readthedocs.io/en/release-1.13/>)
- [8] Corum Security Article (<http://www.corumsecure.com/>)

Experiment Number: 2

Title: To generate SRS (Software Requirements Specification) document for Smart Door Lock System

Pre requisites: Process Models, Engineering and Modelling Practices, Requirements Engineering and Analysis Model

Learning Objectives (Bloom's Taxonomy): Create

Theory:

An SRS is 1Software Requirement Specification document. An SRS establishes the basis for agreement between the client and the supplier on what the software product will do. An SRS provides a reference for validation of the final product. A high-quality SRS is a prerequisite to high-quality software. A high quality SRS reduces the development cost.

A good SRS is – Correct, Complete, Unambiguous, Verifiable, Consistent, Ranked for importance and/or stability, Modifiable, Traceable.

The general structure of SRS firmly based on IEEE standards:

Cover Page

Revisions Page

Table of Contents

1. 1 INTRODUCTION

1.1. Product Overview

2. SPECIFIC REQUIREMENTS

2.1. External Interface Requirements

2.1.1. User Interfaces

2.1.2. Hardware Interfaces

2.1.3. Software Interfaces

2.1.4. Communications Protocols

2.2. Software Product Features

2.3. Software System Attributes

2.3.1. Reliability

2.3.2. Availability

2.3.3. Security

2.3.4. Maintainability

2.3.5. Portability

2.3.6. Performance

2.4. Database Requirements

Hardware & Software requirements:

Hardware: Computer lab, Internet connection for literature survey

Software: Libre Office for preparing document

Procedure:

1. Gather the requirements for the mini project through additional literature survey and meeting the stakeholders, if possible.
3. Refer to the IEEE template provided, and prepare the SRS.

Software Requirements Specification

Smart Door Lock System

Version 1.1.0

Created By

Abhijeet Ringe (47)

Kshitij Rao (45)

Amish Tp. (59)

Don Bosco Institute of Technology, Kurla

Created on: 16/03/2021

Table of Contents

Table of Contents

Revision History

1. Introduction

1.1. Product Overview

2. Specific Requirements

2.1. External Interface Requirements

2.1.1. User Interfaces

2.1.2. Hardware Interfaces

2.1.3. Software Interfaces

2.2. Software Product Features

2.3. Software System Attributes

2.3.1. Reliability

2.3.2. Security

2.3.3. Maintainability

2.3.4. Portability

2.3.5. Performance

2.4. Database Requirements

Revision History

Sr no	Revision Name	Last Modified
1	Integrated Chatbot and facial recognition	22/02/2021
0	Web APP hosted on Heroku	12/01/2021

1. Introduction

The concept of 'Smart Houses' is one of the main trends in rise, due to which, there is need for Home Automation system. A system like Door Lock, which is smart enough to unlock/lock the door via facial recognition, is one of the main components of Home Automation. Such a device can even be implemented in offices, to keep a track of the employees and also to restrict access to unknown people. With the wide spread online shopping there need to be an assistant who will guide the delivery boy when you are not in the house to receive the parcel.

1.1. Product Overview

- The smart door lock system is able to lock/unlock the door using facial recognition.
- It keeps the track of people going in and out of the house by storing the log in the database.
- It uses EM lock for locking the door.
- It also includes interactive chatbot which talks with users giving them a personalized experience.
- This door locking system can also be used in industries and offices to keep track of employees who are going in and out of the premises.

2. Specific Requirements

Hardware Components

- Raspberry Pi 4-2GB RAM
- 2 Channel Relay
- Pi Camera 5MP
- Breadboard Half Size
- Electronic Door Strike
- 12V Power Supply
- ISD1820 Voice Recording Module with On Board Mic and Speaker

Software components

Python Modules used:

- OpenCV
- TF-IDF Vectorizer
- Numpy
- Django
- LBPH Classifier

2.1. External Interface Requirements

2.1.1. User Interface

The overall user interface is made as simple and user-friendly as possible.

The WebApp interface is divided into two parts:

- The Admin
- Normal Users

The admin has the access of the pages on the webapp and can manage (i.e. add / remove) other normal users.

The normal users have view unlock and view rights.

2.1.2. Hardware Interface

The hardware interface consists of a screen of minimum resolution 4000x2500. This display screen will be installed inside the house which will act as door-phone.

This screen will also consist of microphone and a voice module which will capture voice data and feed it to the chatbot model.

The screen will be connected to the camera module which will capture the facial data and feed it to the facial recognition model.

2.1.3. Software Interfaces

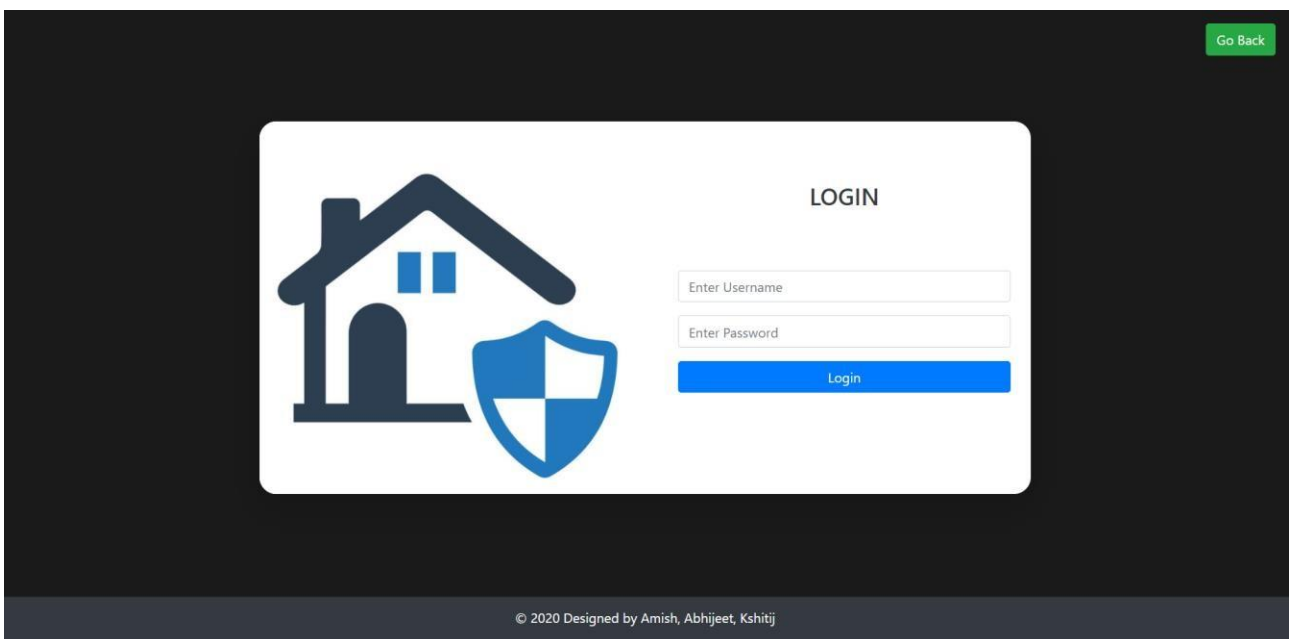
The software interface includes a webapp interface which asks for a login to enter the system.

After the login user has multiple options like add new user ,view logs, watch live footage of the camera, interact with the chatbot, unlock door via OTP and many more.

Each specific user has as different interface according to the access level of that user.

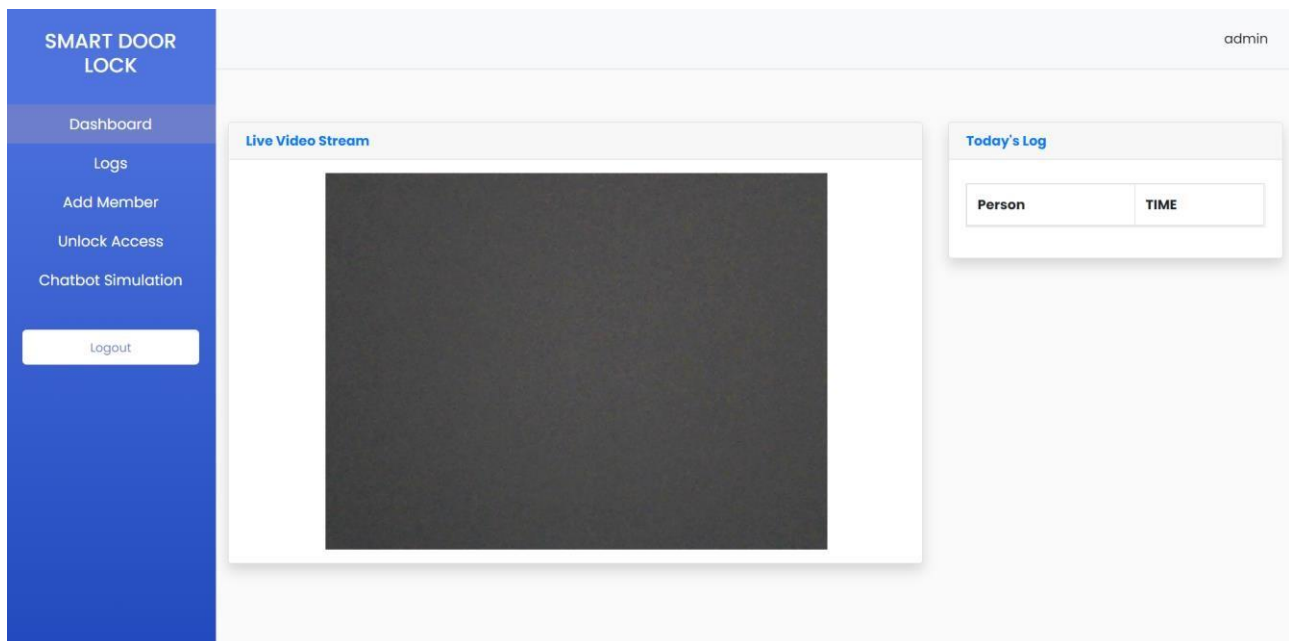
2.2. Software Product Features

The user has to first login to enter the dashboard.



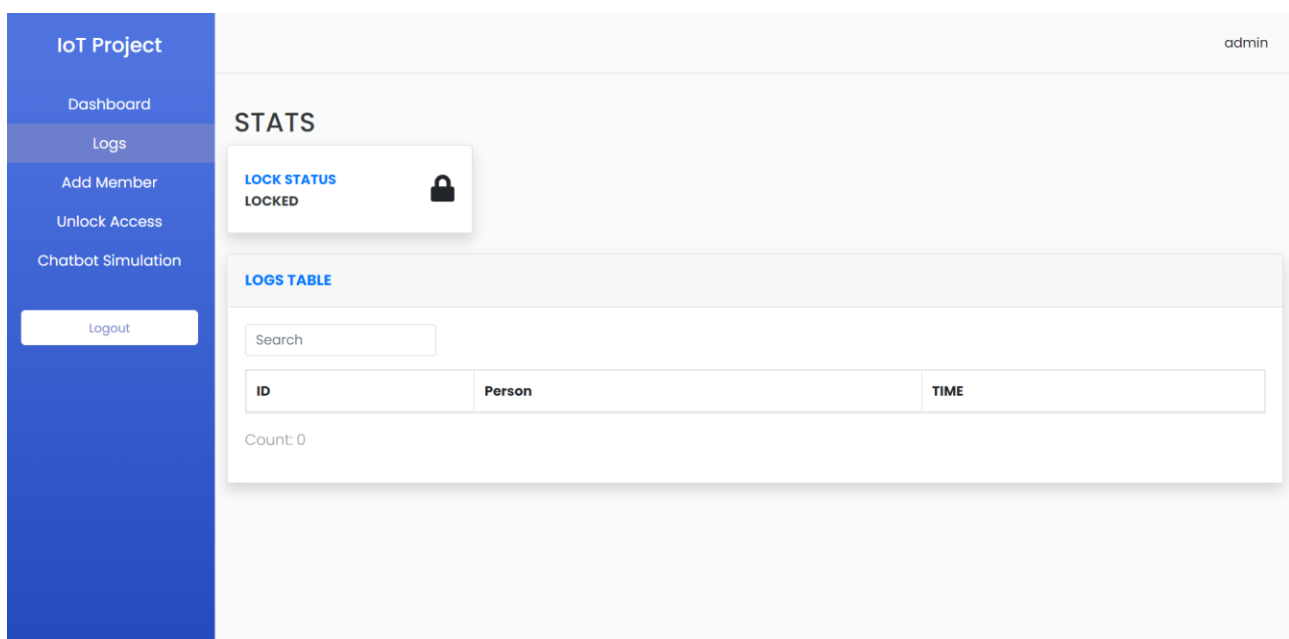
The image shows a login interface for a web application. It features a dark background with a central white login form. The form is titled "LOGIN" and includes a graphic of a house with a shield, symbolizing security. There are two input fields: "Enter Username" and "Enter Password", followed by a blue "Login" button. A green "Go Back" button is located in the top right corner of the form area. At the bottom of the page, there is a footer that reads "© 2020 Designed by Amish, Abhijeet, Kshitij".

After the login part is done, user is redirected to the dashboard screen of the webapp.



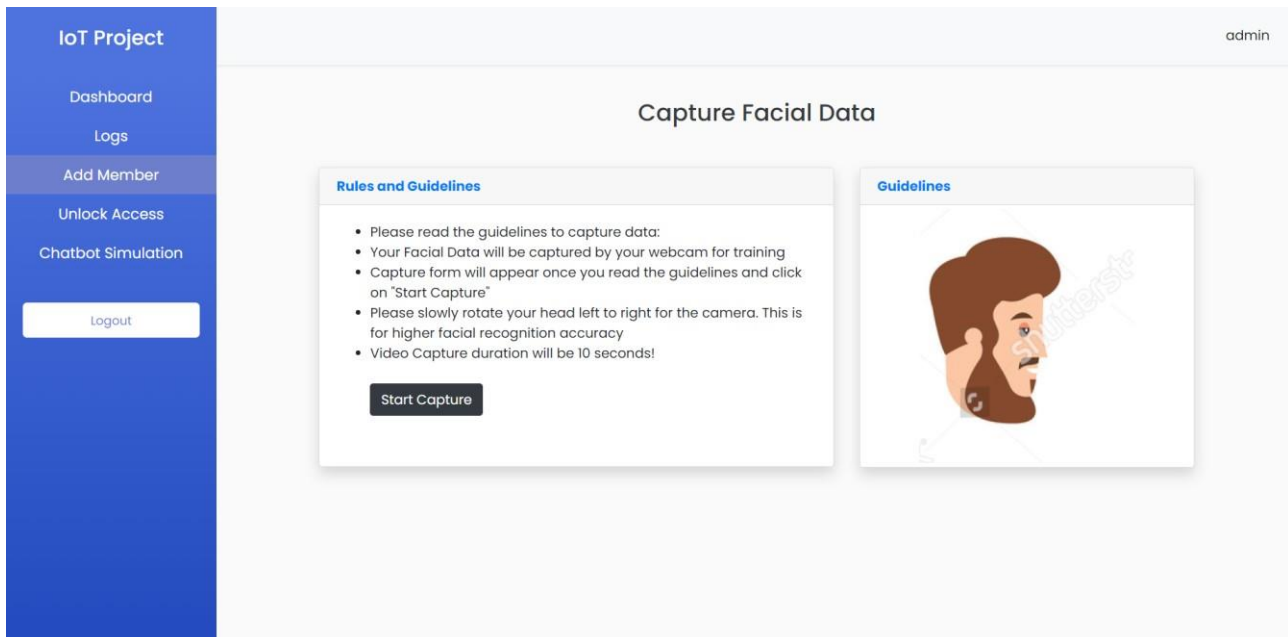
The centre frame shows the live video footage from the camera module. The logs table to the right show the logs for that day.

The logs page look like this



In the add member page, user is first asked to fill the form in which he/she is asked to mention the access level.

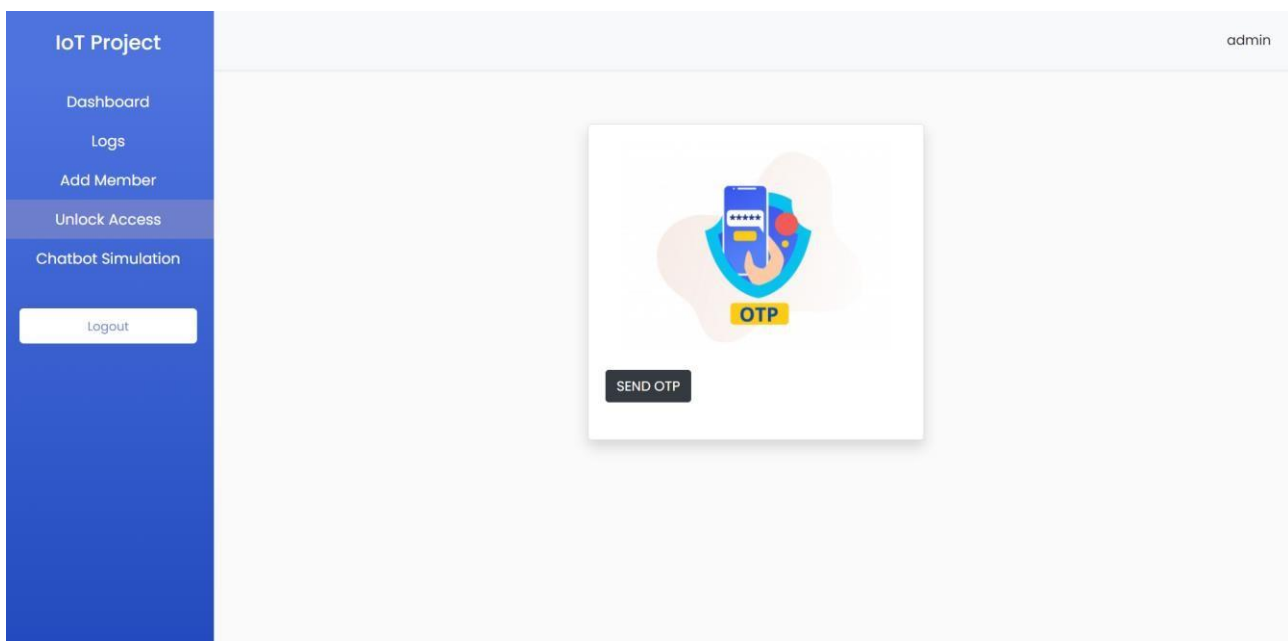
After filling the form the user is redirected to page where the facial data of the user is captured and the user gets registered in the system i.e. the system can recognize the registered user's face.



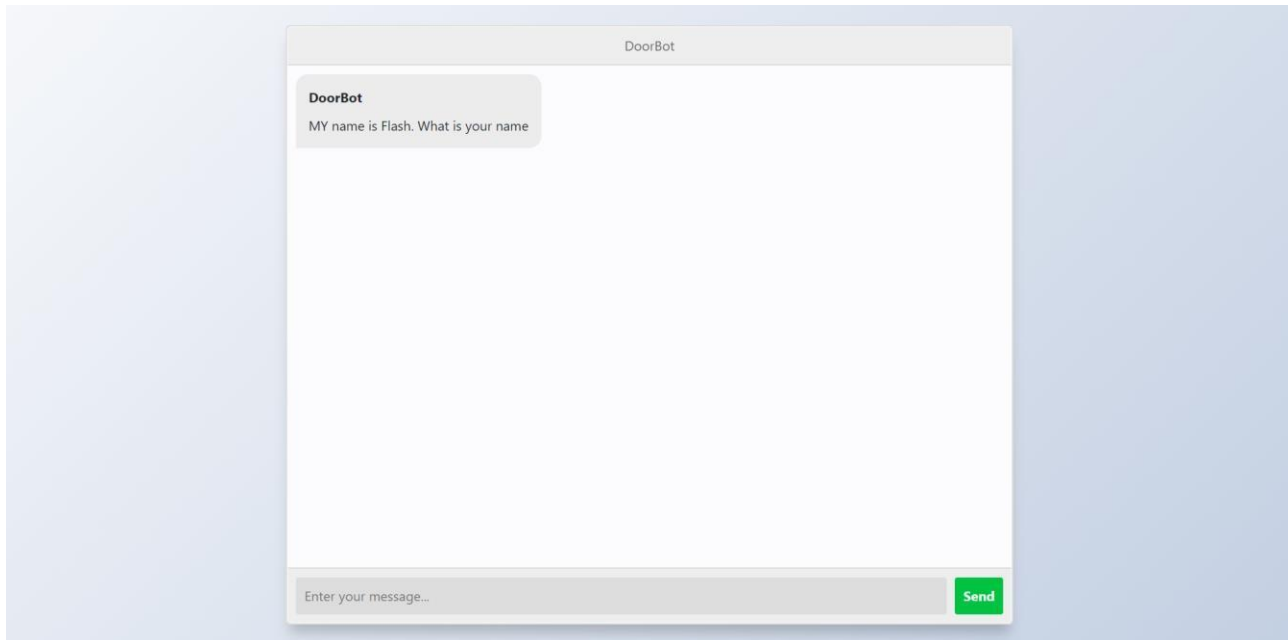
On hitting the start capture button, the camera is turned on and the facial details are captured. The user needs to follow the guidelines mentioned in the right section.

The “unlock access” section is visible only to the view unlock and admin rights.

It includes OTP verification process to unlock the door.



The last section is chatbot where user can interact with an user-friendly chatbot. It helps people guide when nobody is home.



2.3. Software System Attributes

2.3.1. Reliability

Unlike humans, robots are less likely to make mistakes. Thus, this ensures more reliability of these systems. Also, it has OTP system to unlock the door which is accessible only for admin and view unlock access level.

2.3.2. Security

As per a technical survey conducted by CORUM Security, it shows that Facial recognition locks are more secure, compared to other traditional and modern lock systems.

2.3.3. Maintainability

This system only required hardware maintenance and that too once in 3-4 years. The bugs in the software can be fixed by software updates without affecting the hardware alignment.

2.3.4. Portability

This system is very mobile and compact. This is because it has a webapp by which users have the view of their door at any place and any time.

2.3.5. Performance

The facial recognition model gives an accuracy of 80% which can be upgraded to 90+ by a software update. Currently it is using the LBPH model for facial recognition. There are some other techniques as well like triplet loss using facenet which give very high accuracy.

2.4 Database Requirement

This system uses SQLite as the database service. SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. SQLite is the most used database engine in the world. SQLite is built into all mobile phones and most computers and comes bundled inside countless other applications that people use every day. It is also much convenient to integrate it with Django and gives good performance.

Conclusion: We have generated an SRS document for Smart Door Lock System.

References:

1. Software Engineering: A Practitioner's Approach, 6th edition, by Roger S. Pressman
2. An integrated approach to software engineering - Engineering: By Pankaj Jalote (3rd Edition)

Title: To estimate the project using Function point and COCOMO techniques.

Pre requisites: Software Project Estimation-Decomposition Techniques, Function Point Estimation

Learning Objectives (Bloom's Taxonomy): Evaluating

Theory:

1. Function Point Analysis

Since 'functionality' cannot be measured directly, it must be derived indirectly using other direct measures. Function-oriented metrics uses a measure called the function point. Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and assessments of software complexity.

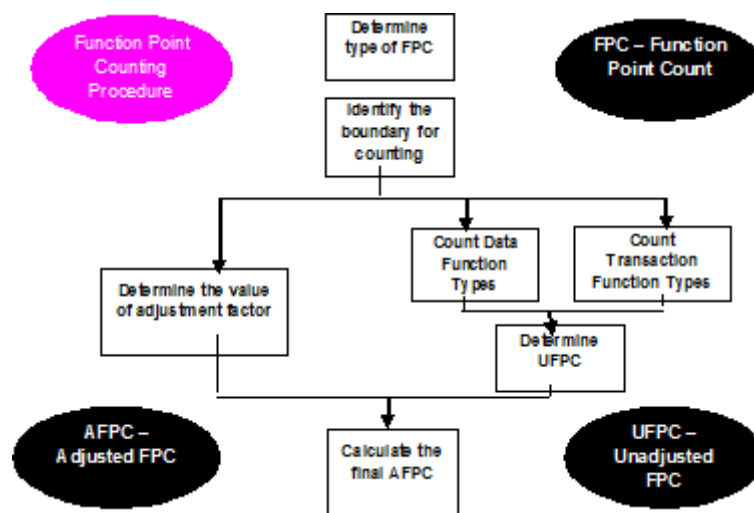


Fig.4. FP Estimation - Procedure

		Weighting factor				
Measurement parameter	Count	Simple Average Complex				
Number of user inputs		×	3	4	6	=
Number of user outputs		×	4	5	7	=
Number of user inquiries		×	3	4	6	=
Number of files		×	7	10	15	=
Number of external interfaces		×	5	7	10	=
Count total						→

Fig.5. Computing function points

Function points are computed by completing the table shown in Fig. 3.2. Five information domain characteristics are determined and counts are provided in the appropriate table location. Information

domain values are defined in the following manner:

Number of user inputs: Each user input that provides distinct application oriented data to the software is counted. Inputs should be distinguished from inquiries, which are counted separately.

Number of user outputs: Each user output that provides application-oriented information to the user is counted. In this context output refers to reports, screens, error messages, etc. Individual data items within a report are not counted separately.

Number of user inquiries: An inquiry is defined as an on-line input that results in the generation of some immediate software response in the form of an on-line output. Each distinct inquiry is counted.

Number of files: Each logical master file (i.e., a logical grouping of data that may be one part of a large database or a separate file) is counted.

Number of external interfaces: All machine-readable interfaces (e.g., data files on storage media) that are used to transmit information to another system are counted.

Once these data have been collected, a complexity value is associated with each count.

Organizations that use function point methods develop criteria for determining whether a particular entry is simple, average, or complex. Nonetheless, the determination of complexity is somewhat subjective.

To compute function points (FP), the following relationship is used:

$$\text{FP} = \text{count total} * [0.65 + 0.01 * \Sigma(\text{Fi})] \text{-----} (1)$$

Where, count total is the sum of all FP entries obtained from Fig. 1.2.

The F_i ($i = 1$ to 14) are "complexity adjustment values" based on responses to the following questions:

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
- Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Each of these questions is answered using a scale that ranges from 0 (not important or applicable) to 5 (absolutely essential). The constant values in Equation (1) and the weighting factors that are applied to information domain counts are determined empirically.

Then using Table 1, calculate the size of the mini project as,

$$\text{LOC} = \text{FP} * (\text{LOC/FP}) \text{-----} (2)$$

Language	Average Source LOC per Function Pont	Average Source LOC for a 210 FP Application
Access	38	7,980
Basic	107	22,470
C	128	26,880
C++	53	11,130
COBOL	107	22,470
Delphi	29	6,090
Java	53	11,130
Machine Language	640	134,440
Visual Basic 5	29	6,090

Source: <http://www.theadvisors.com/langcomparison.htm>

Table 1: Average source LOC per function point

3. COCOMO

COCOMO is a cost estimation model the model of COCOMO I is also called COCOMO'81 is presented. Parametric Model developed by Barry Boehm in 1981 .Boehm proposed three levels of the model: basic, intermediate, detailed.

- The basic COCOMO model is a single-valued, static model that computes software development effort (and cost) as a function of program size expressed in estimated thousand delivered source instructions (KDSI).
- The intermediate COCOMO model computes software development effort as a function of program size and a set of fifteen "cost drivers" that include subjective assessments of product, hardware, personnel, and project attributes.
- The advanced or detailed COCOMO model incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step (analysis, design, etc.) of the software engineering process.

COCOMO models depends on the two main equations

1. Development effort: $MM = a * KDSI^b$

Based on MM - man-month / person month / staff-month is one month of effort by one person. In COCOMO'81, there are 152 hours per Person month. According to organization this values may differ from the standard by 10% to 20%.

2. Effort and development time (TDEV): $TDEV = 2.5 * MM^c$

The coefficients a, b and c depend on the mode of the development. There are three modes of Development:

Development Mode	Project Characteristics			
	Size	Innovation	Deadline/constraints	Dev. Environment
Organic	Small	Little	Not tight	Stable
Semi-detached	Medium	Medium	Medium	Medium
Embedded	Large	Greater	Tight	Complex hardware/ customer interfaces

Fig.6. Development Modes

The basic COCOMO applies the parameterised equation without much detailed consideration of project characteristics.

	Basic COCOMO	a	b	c
$MM = a * KDSI^b$	Organic	2.4	1.05	0.38
	Semi-detached	3.0	1.12	0.35
$TDEV = 2.5 * MM^c$	Embedded	3.6	1.20	0.32

Fig.7. Equations and Parameters of the basic COCOMO

ADVANTAGES OF COCOMO

COCOMO is transparent; one can see how it works.

DRAWBACKS OF COCOMO

- It is hard to accurately estimate KDSI early on in the project, when most effort estimates are required.
- KDSI, actually, is not a size measure it is a length measure.
- Extremely vulnerable to mis-classification of the development mode.
- Success depends largely on tuning the model to the needs of the organization, using historical data which is not always available.

Hardware & Software requirements:

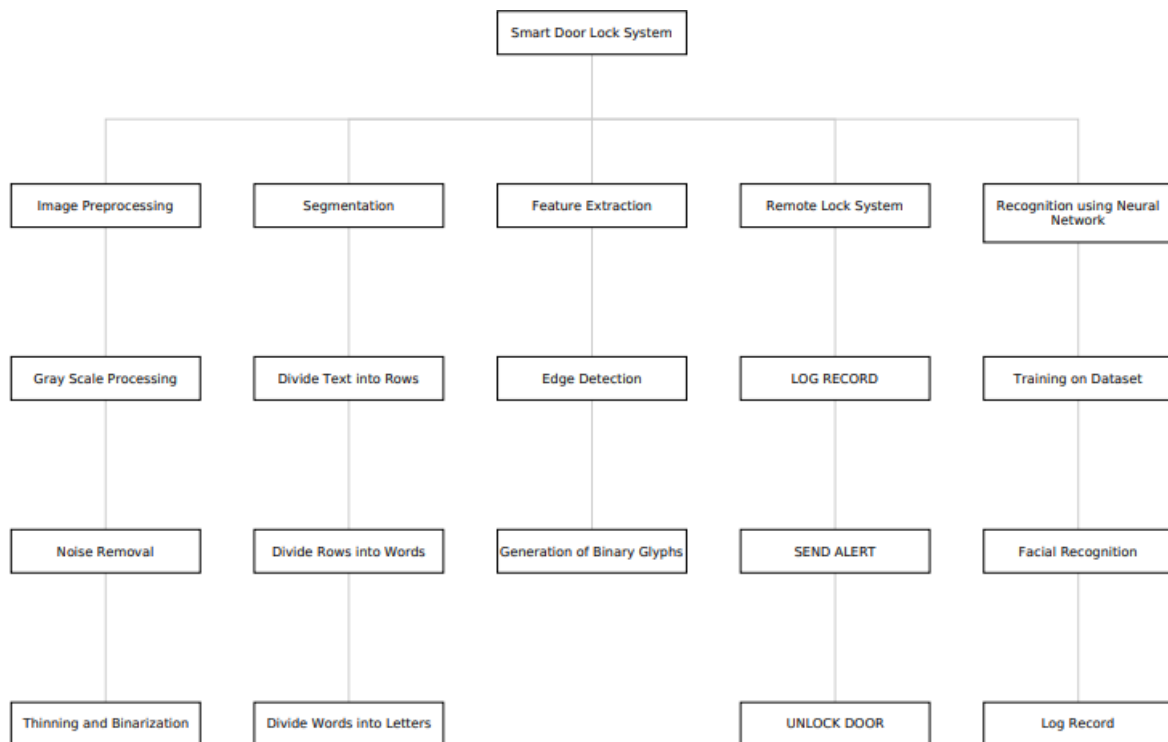
Hardware: Computer lab, Internet connection for literature survey

Software: Libre Office for preparing document

Procedure:

Procedure for creating WBS

1. Create the work Package for the Mini project
2. Develop the WBS according to the work package in any format (either Graphical or Outline)



Procedure for FPA

1. Breakdown the project logically into functional modules.

Module 1: Preprocessing

Gray Scale:

An image is an array, or a matrix, of square pixels (picture elements) arranged in columns and rows. In an (8 bit) gray scale image each picture element has an assigned intensity that ranges from 0 to 255. A gray scale image is what people normally call a black and white image, but the name emphasizes that such an image will also include many shades of gray. A normal gray scale image has 8 bit color depth- 256 gray scales. A “true color images 24 bit color depth $8 * 8 * 8$ bits $256 * 256 * 256$ colors = 16 million colors. Some gray scale images have more gray scales, for instance 16 bit = 65536 gray scales. There are two general groups of images: vector graphics (or line art) and bitmaps (pixel based or images).

Thinning:

Thinning algorithm is a Morphological operation that is used to remove selected foreground pixels from binary images. It preserves the topology (extent and connectivity) of the original region while throwing away most of the original foreground pixels.

Module 2: Segmentation

In this part the characters will be identified as letters and the image will be converted to text. After the image is cleaned up and becomes a binary image which contains only the text, the binary image is then saved and the memory is cleaned up. This step is very important to increase the speed of the system. After the following steps should be done.

- Divide the text into rows
- Divide the rows into words
- Divide the word into letters

Module 4: Feature Extraction

In remote lock system, we will have to ensure that before the unlocking happens, we need to log the person's data and then verify if the person is authorised to enter the place. If he is, then the system unlocks, and allows the person to enter.

Module 4: Feature Extraction

In pattern recognition in image processing, Feature extraction is a special form of dimensionality reduction. Edge Detection and Binary Glyphs Generation are two main features. Edge Detection works by detecting discontinuities in brightness.

When the input data to an algorithm is too large to be processed and it is suspected to be notoriously redundant (much data, but not much information) then the input data will be transformed into reduced representation set of features (also named feature vector). Transforming the input data into the set of features is called features extraction.

Module 5: Recognition Using Neural Network

The back end for recognition is the neural network designed using Deep Learning Classification Algorithms. The number of input to each network is associated with the size of the feature vector for each text. The no of output considered for this data sets are 27(Characters a-z and 1 reject neuron for non-character pattern).

2. Determine the information domain values for the mini project.
3. Prepare the table shown in Fig.5 and compute the count total.

Measurement Parameter	Count	Simple	Average	Complex	
No. of user inputs	1	3	4	6	4
No. of user outputs	1	4	5	7	5
No. of user inquiries	1	3	4	6	4
No. of files	1	7	10	15	10
No. of external interfaces	1	5	7	10	7
Count Total					30

4. Calculate F_i ($i = 1$ to 14), the "complexity adjustment values", based on responses to the above fourteen questions.
5. Calculate the function points using the Eqn. (1).

$$FP = 30 * [0.65 + 0.01 * 37]$$

$$FP = 30.6$$

6. Estimate the size in LOC using Table 1 and Eqn. (2).

$$LOC = 30.6 * (53/30.6)$$

$$LOC = 53$$

Procedure for COCOMO

Gather the size of mini project (in LOC) obtained reconciling FP and LOC estimations.
Determine the type of the mini project using Fig.6

Calculate effort from size in LOC using the relevant equation of CoCoMo, using Fig. 7, based on type of project.

Calculate duration from effort using the relevant equation of CoCoMo, using Fig. 7, based on type of project.

Calculate people required as duration/effort and hence the labour cost using the equation

People Required = Effort / Duration

Estimated kilo line of codes: 53

Effort = $a * loc^b$

Duration = $c * effort^d$

Staffing = effort/duration

	Organic	Semi-detached	Embedded
Variable A	2.4	3	3.6
Variable B	1.05	1.12	1.2
Variable C	2.5	2.5	2.5
Variable D	0.38	0.35	0.32
KLOC	53	53	53
Effort (In Person / Month)	155.13182612468924	256.04142091653273	422.117681690283
Duration (In months)	16.99833376749732	17.4119972024207	17.301300242855472
Staffing (Recommended)	9.126296038575164	14.704885254687303	24.398032273013435

Conclusion: The project estimation was completed using FP and COCOMO techniques.

References:

1. "Information Technology Project Management", Jack T. Marchewka, 4th edition, Wiley India, 2014.
2. <http://www.projectmanagementdocs.com/project-planning-templates>

Experiment Number: 4

Title: To perform Structural modeling through UML diagrams (Object Diagram, Class Diagram, Component diagram and Deployment diagram) for the smart door lock system

Pre requisites: SRS, system information and its flow.

Tool used: Dia tool

Learning Objectives (Bloom's Taxonomy): Creating

Theory:

The Unified Modeling Language (UML) is a language for - Visualizing - Specifying – Constructing - Documenting the system. The UML is a notation and a semantic language applied to any software development process. UML diagrams represent two different views of a system model: -

Static (or structural) view: emphasizes the static structure of the system using objects, attributes, operations and relationships. The structural view includes class diagrams and composite structure diagrams.

Dynamic (or behavioral) view: emphasizes the dynamic behavior of the system by showing collaborations among objects and changes to the internal states of objects. This view includes sequence diagrams, activity diagrams and state machine diagrams.

UML models can be exchanged among UML tools by using the XML Metadata Interchange (XMI) interchange format.

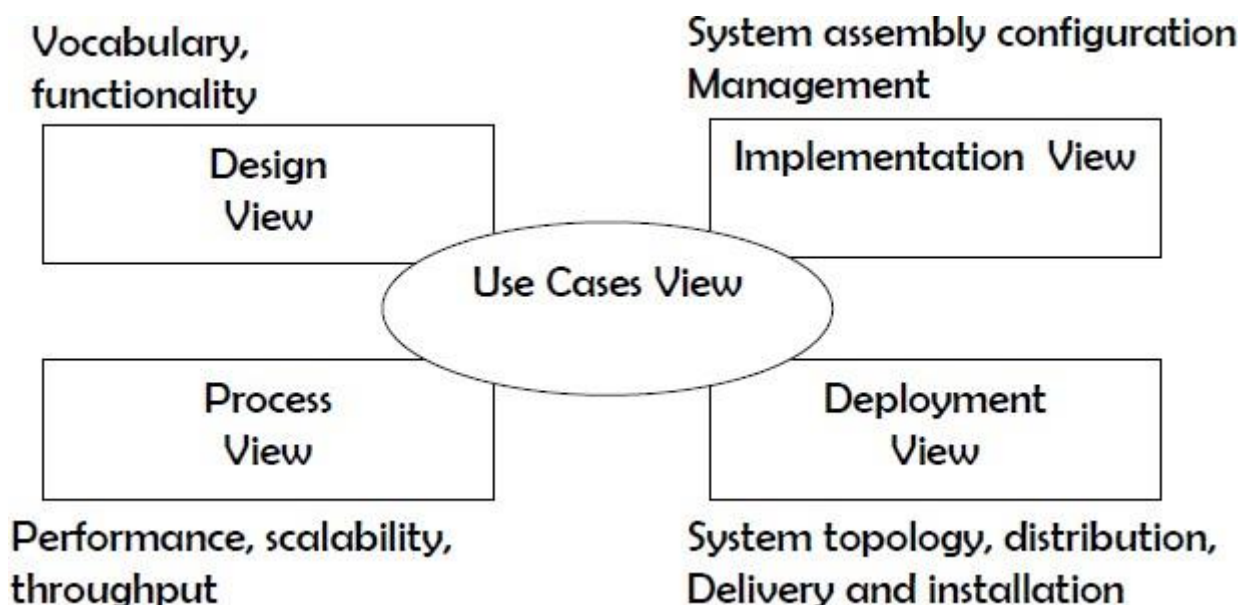


Fig.1. UML Architecture

The UML Diagrams

UML 2 has many types of diagrams which are divided into two categories[4].

Some types represent structural information, and the rest represent general types of behavior, including a few that represent different aspects of interactions. These diagrams can be categorized hierarchically as shown in the following class diagram.

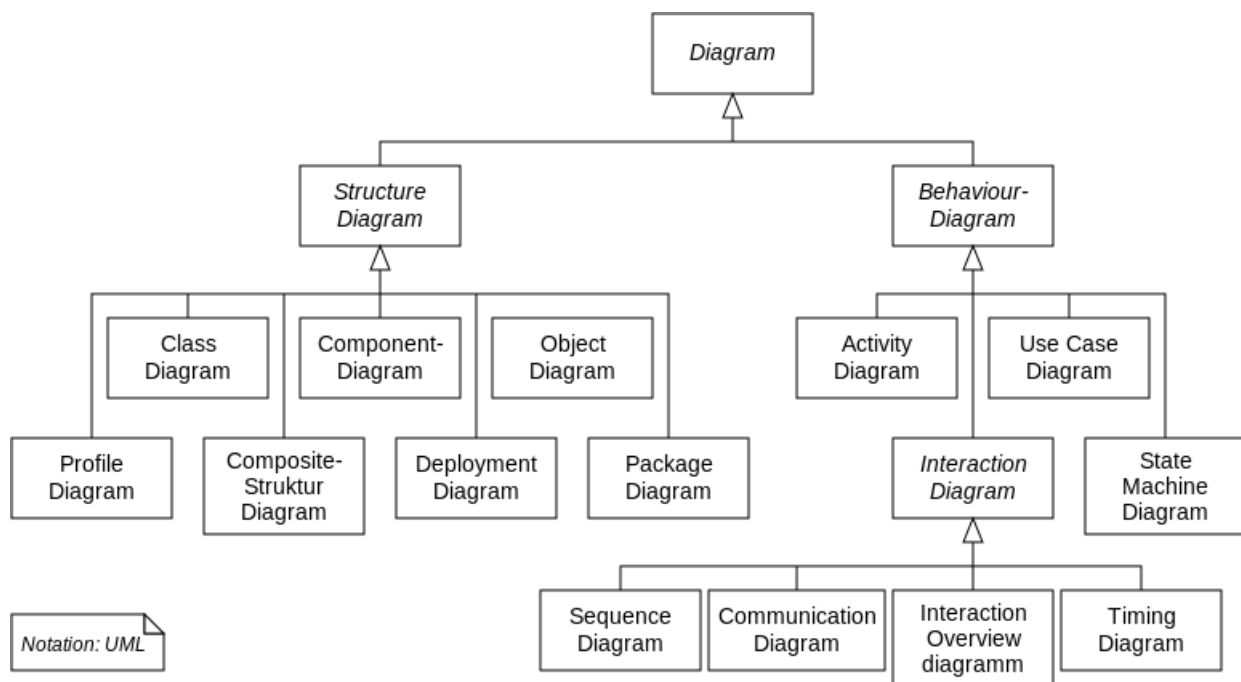


Fig.2. Hierarchy of UML Diagrams

These diagrams may all contain comments or notes explaining usage, constraint, or intent.

Structure diagrams

Structure diagrams emphasize the things that must be present in the system being modeled. Since structure diagrams represent the structure, they are used extensively in documenting the software architecture of software systems. For example, the component diagram which describes how a software system is split up into components and shows the dependencies among these components.

Behavior diagrams

Behavior diagrams emphasize what must happen in the system being modeled. Since behavior diagrams illustrate the behavior of a system, they are used extensively to describe the functionality of software systems. As an example, the activity diagram describes the business and operational step-by-step activities of the components in a system.

Interaction diagrams

Interaction diagrams, a subset of behavior diagrams, emphasize the flow of control and data among

the things in the system being modeled. For example, the sequence diagram which shows how objects communicate with each other in terms of a sequence of messages.

Hardware & Software requirements:

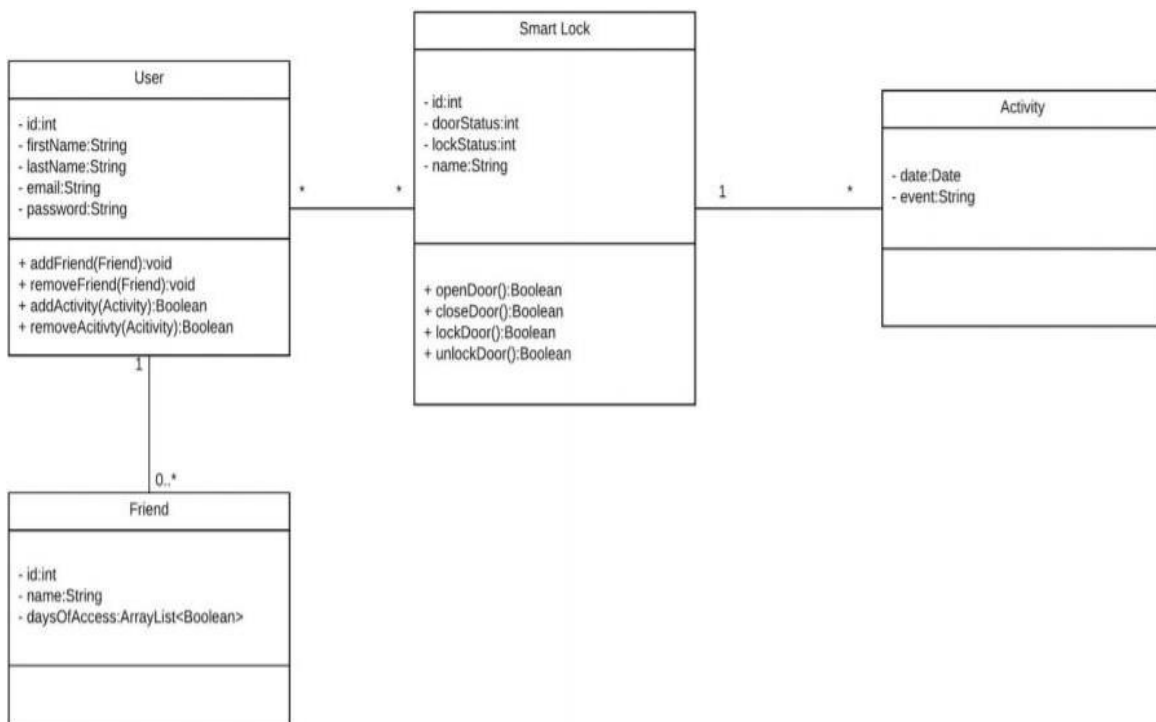
Hardware: Computer lab, Internet connection for literature survey

Software: Libre Office for preparing document

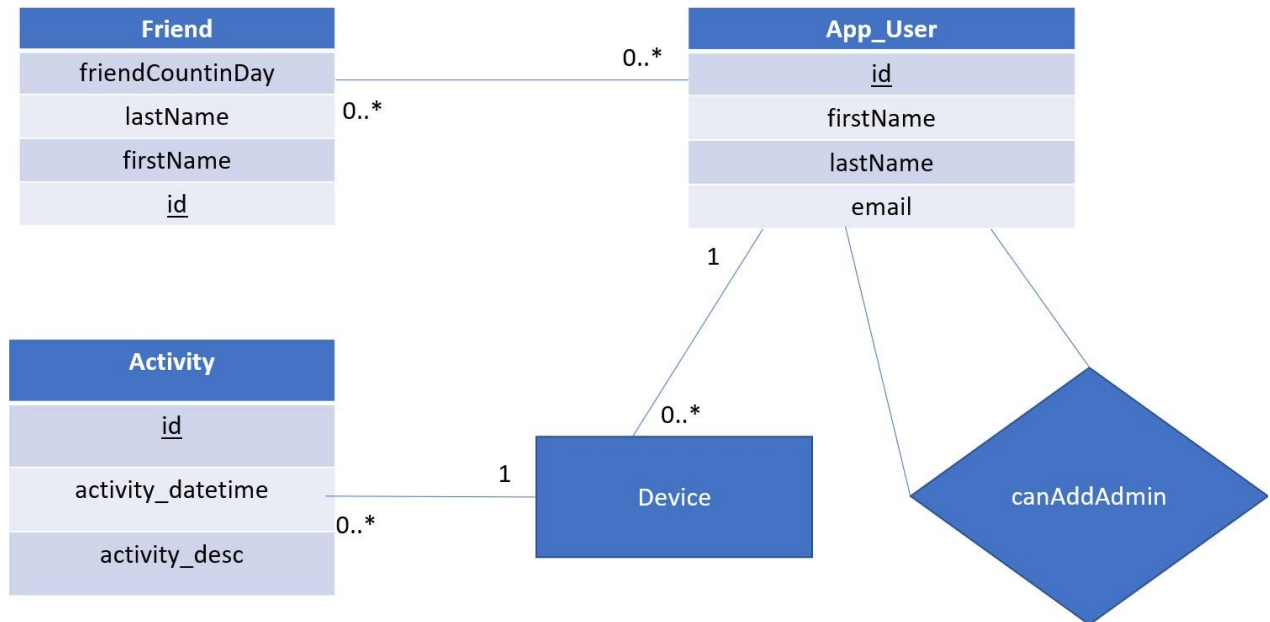
Procedure:

1. Draw the various UML (Class, Object, Component, Deployment diagrams using any open source tool)

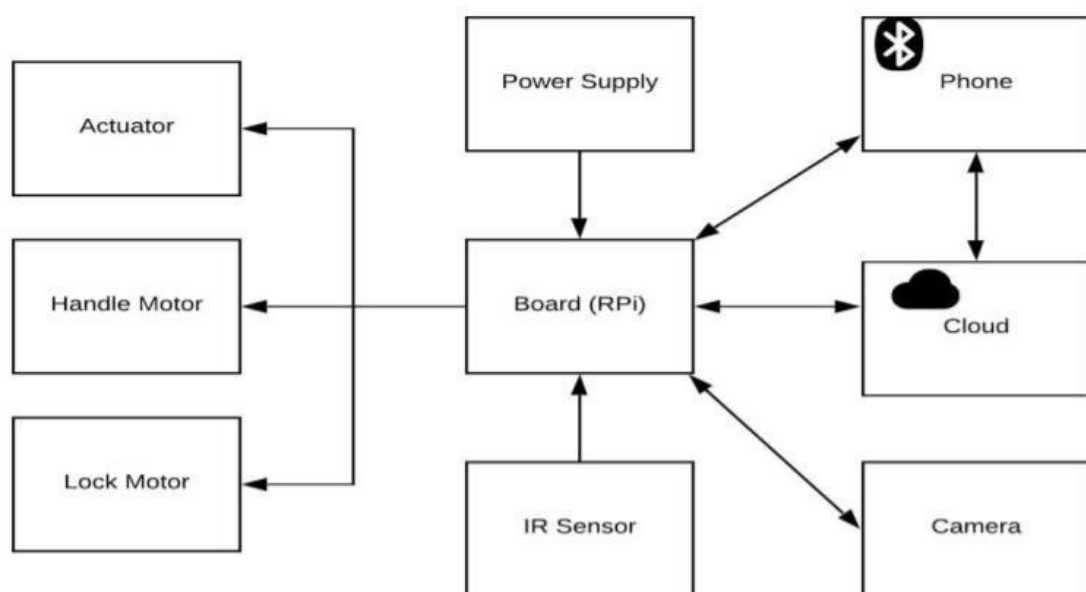
Class Diagram



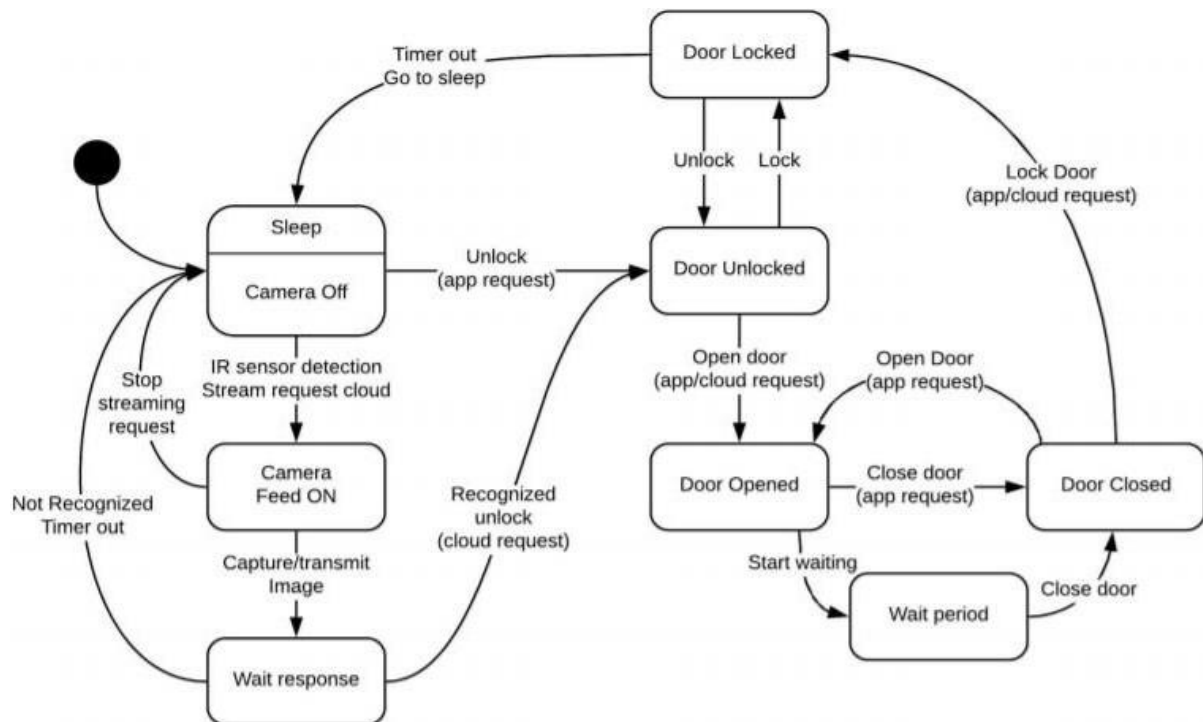
Object Diagram



Component Diagram



Deployment Diagram



Conclusion:

We gained knowledge about Structural Modelling through different types of UML diagrams like Object Diagram, Class Diagram, Component Diagram, and Deployment Diagram for smart door lock system.

References:

1. Software Engineering: A Practitioner's Approach, 6th edition, by Roger S. Pressman
2. Document templates for Student projects in Software engineering, Declan Delaney and Stephen Brown, Department of Computer Science, National University of Ireland, Maynooth, Technical report: nuim-cs-tr2002-05
3. Jon Holt Institution of Electrical Engineers (2004). UML for Systems Engineering: Watching the

Wheels IET, 2004, ISBN 0-86341-354-4. p.58

4. Satish Mishra (1997). "Visual Modeling & Unified Modeling Language (UML): Introduction to UML". Rational Software Corporation. Accessed 9 November 2008.

Experiment Number: 5

Behavioural Modeling-1

Title: To perform Behavioural modelling through UML diagrams (Use- Case Diagram, Sequence Diagram and Collaboration Diagram)

Pre requisites: SRS, system information and its flow.

Tool used: Dia tool

Learning Objectives (Bloom's Taxonomy): Creating

Theory:

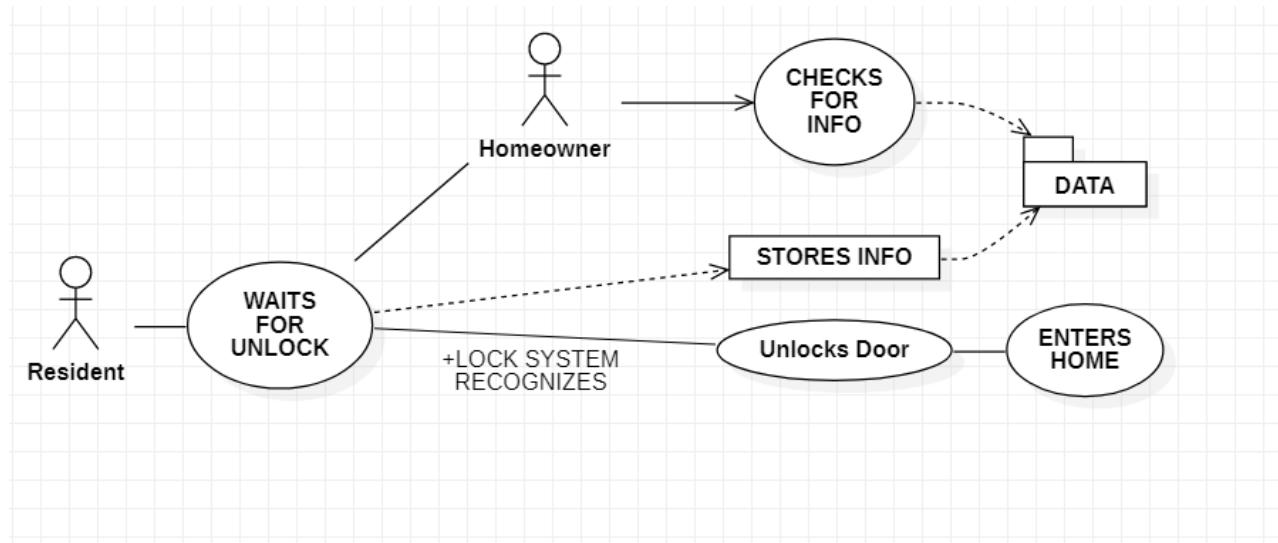
A **Use case diagram** is the primary form of system/software requirements for a new software program underdeveloped. Use cases specify the expected behaviour (what), and not the exact method of making it happen (how).

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.

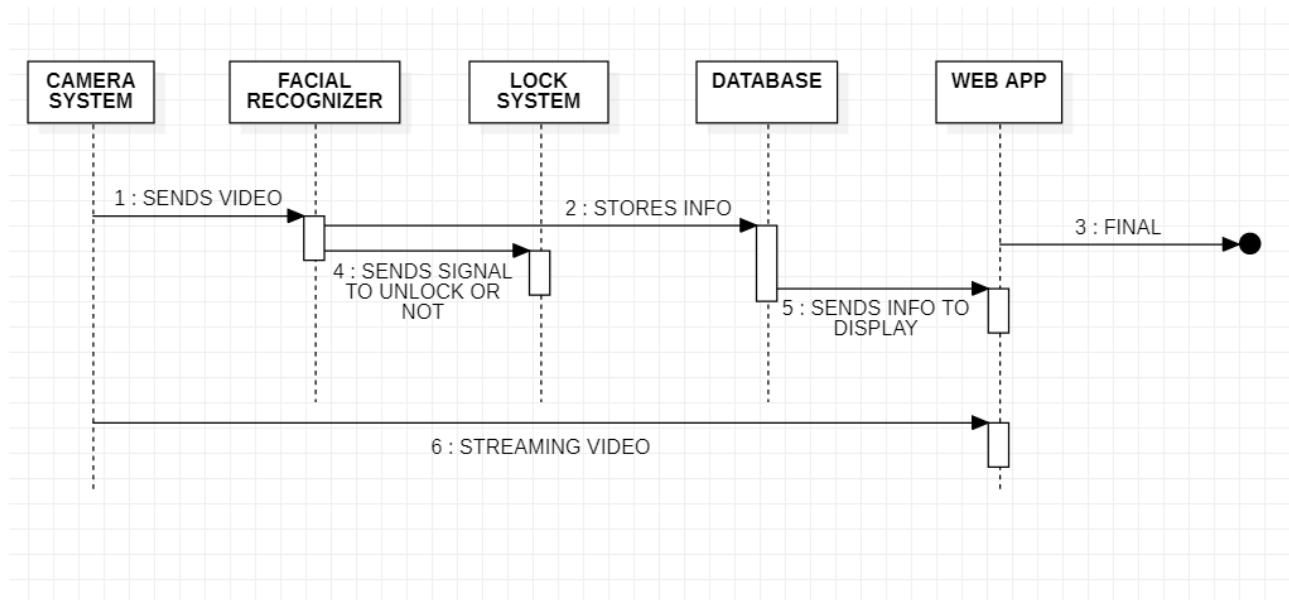
Procedure:

1. Draw the various Behavioural UML diagrams (Use- Case Diagram, Sequence Diagram and Collaboration Diagram)

USE-CASE DIAGRAM



SEQUENCE DIAGRAM



Conclusion:

We can now confidently design Use Case Diagram and Sequence Diagram, for the mini project. The fundamentals behind designing such diagrams is clear now, and these diagrams help in understanding better of the problem statement.

References:

1. Software Engineering: A Practitioner's Approach, 6th edition, by Roger S. Pressman
2. Document templates for Student projects in Software engineering, Declan Delaney and Stephen Brown, Department of Computer Science, National University of Ireland, Maynooth, Technical report: nuim-cs-tr2002-05
3. Jon Holt Institution of Electrical Engineers (2004). UML for Systems Engineering: Watching the Wheels IET, 2004, ISBN 0-86341-354-4. p.58
4. Satish Mishra (1997). "Visual Modeling & Unified Modeling Language (UML): Introduction to UML". Rational Software Corporation. Accessed 9 November 2008.

Experiment Number: 6 Behavioral Modeling-2

Title: To perform Behavioural modelling through UML diagrams (Activity and State chart Diagram)

Pre requisites: SRS, system information and its flow.

Tool used: Dia tool or Star UML

Learning Objectives (Bloom's Taxonomy): Creating

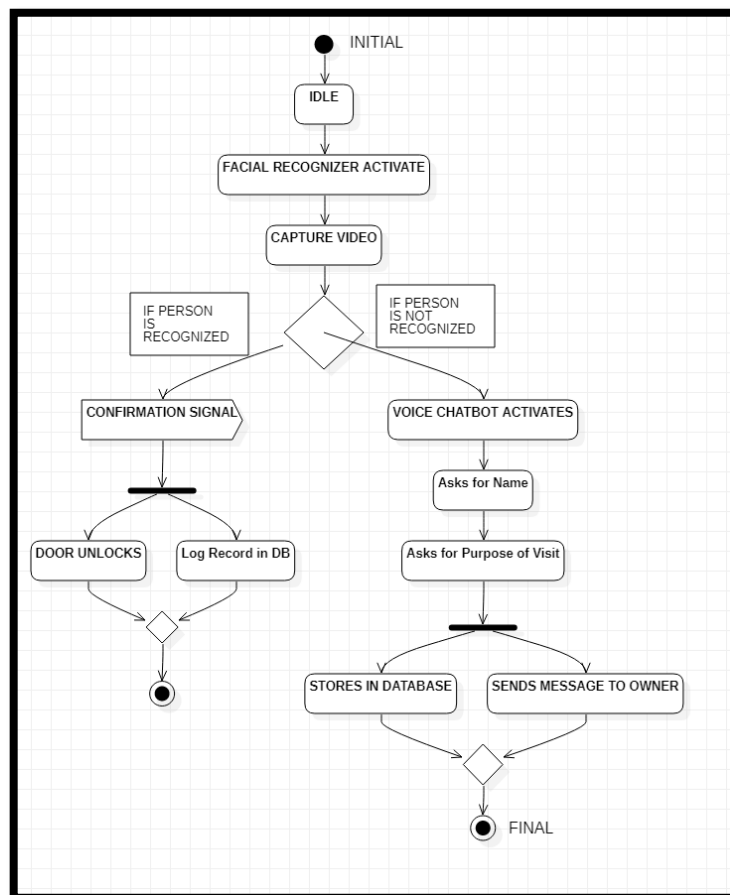
Theory:

Behavioral (or Dynamic) view: emphasizes the dynamic behavior of the system by showing collaborations among objects and changes to the internal states of objects. This view includes sequence diagrams, activity diagrams, and state machine diagrams.

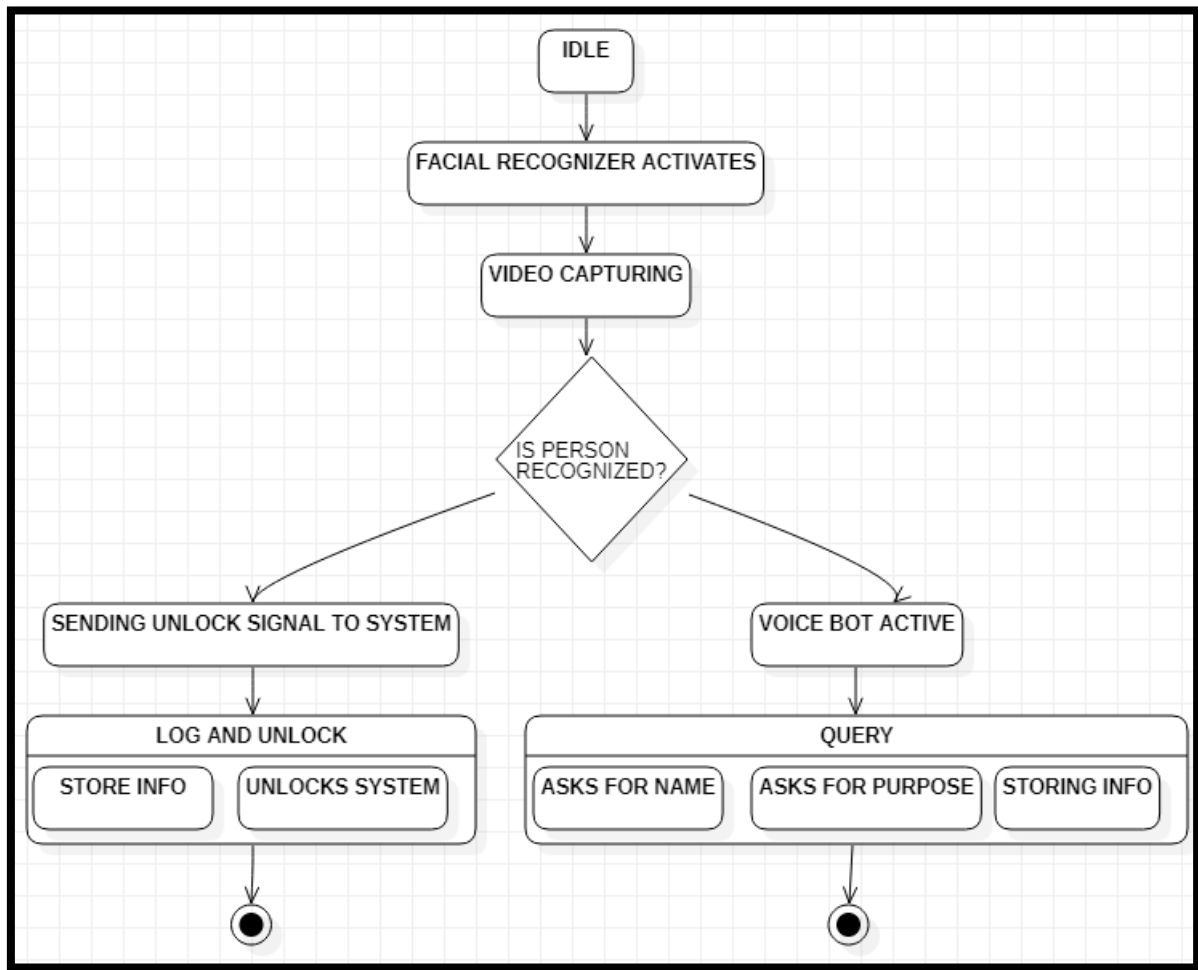
Procedure:

Draw the various Behavioural UML diagrams (Activity and state chart Diagram).

ACTIVITY DIAGRAM



STATE CHART DIAGRAM



Conclusion:

I now completely understand, what are the components of Activity Diagram and State Chart Diagram, and design the Behavioural UML Diagrams as per the Mini Project, that I have chosen.

References:

1. Software Engineering: A Practitioner's Approach, 6th edition, by Roger S. Pressman
2. Document templates for Student projects in Software engineering, Declan Delaney and Stephen Brown, Department of Computer Science, National University of Ireland, Maynooth, Technical report: nuim-cs-tr2002-05
3. Jon Holt Institution of Electrical Engineers (2004). UML for Systems Engineering: Watching the Wheels IET, 2004, ISBN 0-86341-354-4. p.58
4. Satish Mishra (1997). "Visual Modeling & Unified Modeling Language (UML): Introduction to UML". Rational Software Corporation. Accessed 9 November 2008.

Experiment Number: 7

Software Design Document

Title: To create the software design document for the case study “Smart Door Lock”

Pre requisites: SRS, system information and its flow.

Learning Objectives (Bloom’s Taxonomy):

A software design document—sometimes called software design specification—is a detailed plan for developing a piece of software. An SDD should outline the finished software’s functionality (specs) and your team’s plans to build it (timeline, goals, etc.).

Whether you’re an independent contractor or a project manager at a large company, this much stays true: For any project that will take longer than a month, you should create an SDD before you start coding. It’s tempting to skip the planning steps and get right into coding. After all, software design documents aren’t exactly fun to make. But trust us, taking the time to create a good SDD will save you from future headaches. Software design documents break down nearly every aspect of a software development project: What problem will the software solve? What will the final product look like? How does the internal architecture function? (We’ll get into the specifics of what to include in your SDD later.)

Procedure:

- 1 INTRODUCTION
 - 1.1 Project Overview
 - 1.2 Project Deliverables
- 2 PROJECT ORGANIZATION
 - 2.1 Software Process Model
 - 2.2 Roles and Responsibilities
 - 2.3 Tools and Techniques
- 3 PROJECT MANAGEMENT PLAN
 - 3.1 Tasks
 - 3.1.n Task-*n*
 - 3.1.n.1 Description
 - 3.1.n.2 Deliverables and Milestones
 - 3.1.n.3 Resources Needed
 - 3.1.n.4 Dependencies and Constraints
 - 3.1.n.5 Risks and Contingencies
 - 3.2 Assignments
 - 3.3 Timetable
- 4 ADDITIONAL MATERIAL

1.1 Project Overview

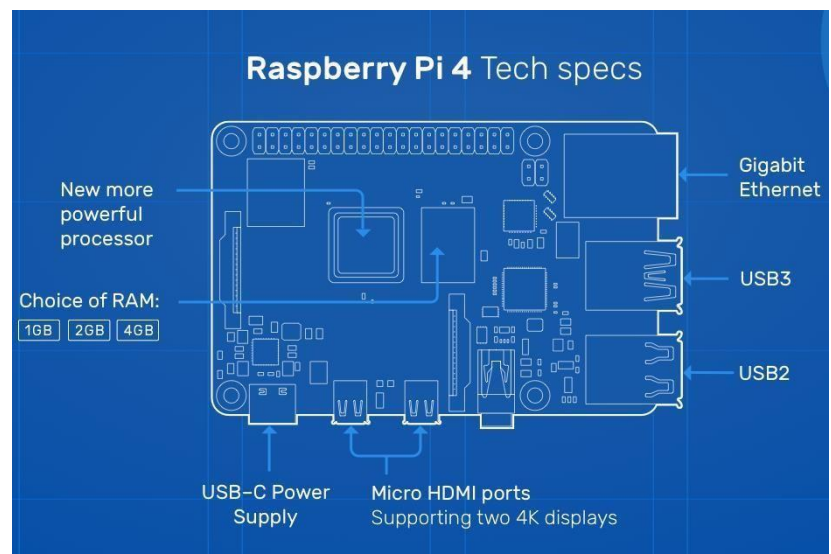
- **DOMAIN** : IoT (Internet of Things)
- **Project Description:**
 - Smart Door Lock System with Facial Recognition and Voice Assistant
 - System Interface hosted on Django Server and designed using HTML, CSS, Bootstrap 4 etc.
 - Backend Database : **SQLite**
- **Website Functionalities:**
 - Live Video Streaming from the Lock System, displayed in the website
 - Display Visitor Logs from Database Data
 - Member Registration with Facial Data Capture for Training Facial Recognizer
 - Sending unlock request over Internet, after OTP Verification
 - Authorization Level Based Access to Interface
 - To show the conversational workflow of the Voice Assistant in the Website
- **Python Modules used** [\[7\]](#):
 - OpenCV
 - Numpy
 - LBPH Classifier
 - TF-IDF Vectorizer
 - Django

2.1 System Architecture/design:

About Raspberry Pi 4:

Raspberry Pi 4 is a single board ARM Based computer. Below are its specifications.

- Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
- 2GB LPDDR4-3200 SDRAM
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE
- Gigabit Ethernet
- 2 USB 3.0 ports; 2 USB 2.0 ports.
- Raspberry Pi standard 40 pin GPIO header (fully backwards compatible with previous boards)



- 2 × micro-HDMI ports (up to 4kp60 supported)
- 2-lane MIPI DSI display port
- 2-lane MIPI CSI camera port
- 4-pole stereo audio and composite video port
- H.265 (4kp60 decode), H264 (1080p60 decode, 1080p30 encode)
- OpenGL ES 3.0 graphics
- Micro-SD card slot for loading operating system and data storage
- 5V DC via USB-C connector (minimum 3A*)
- 5V DC via GPIO header (minimum 3A*)
- Power over Ethernet (PoE) enabled (requires separate PoE HAT)
- Operating temperature: 0 – 50 degrees C ambient

3. Hardware Components

- Raspberry Pi 4-**2GB** RAM
- 2 Channel Relay
- Pi Camera 5MP
- Breadboard Half Size
- Electronic Door Strike
- 12V Power Supply
- ISD1820 Voice Recording Module With On Board Mic and Speaker

Explanation:

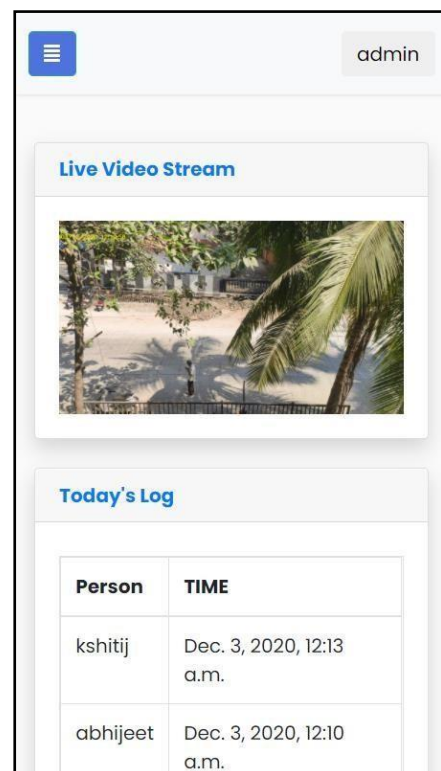
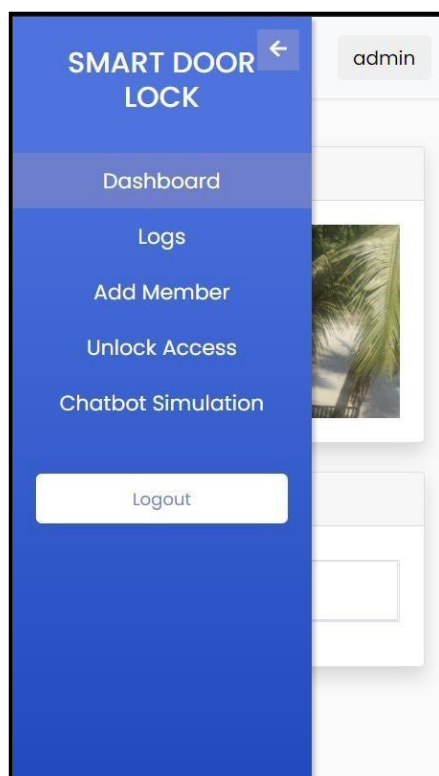
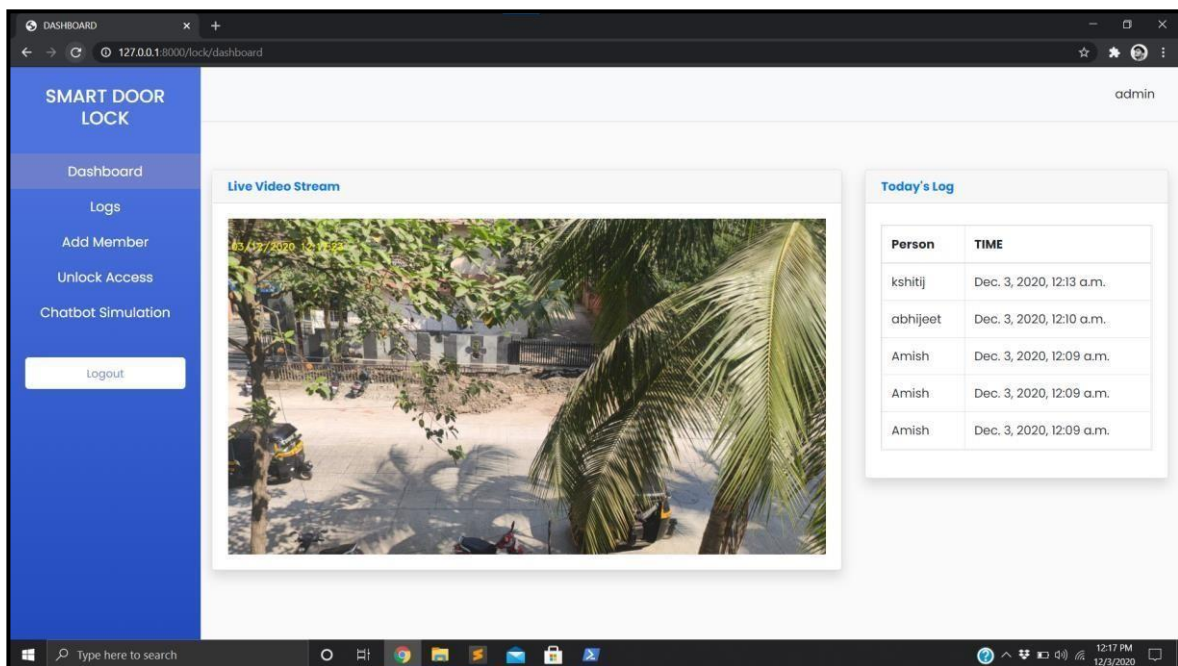
Raspberry Pi [\[7\]](#) serves as the backbone of this smart door lock system. All the programming takes place in it, and using the GPIO Pins on the Raspberry Pi, we send and receive data/signals from other parts of the system.

A Solenoid Lock is perfect for the lock mechanism. The 2 Channel Relay helps in sending/receiving signals with the Solenoid Lock. Status of the lock can then be viewed in the Web App.

Video is captured through Raspberry Pi Camera and it is send to two different parts of the program, one for detection and one for streaming on the Web App. The Whole system is powered up by a Power Supply of 12V. In case of failure, the system will switch to Battery Pack for operation.

The above whole circuit would be placed in an alloy material enclosure to prevent tampering or damage to systems.

4 User Interface:



4.1 HTML5, CSS3

Below is the base layout of web site code and preview.

HTML Code:

```
<body>
  <div class="wrapper">
    <!-- Sidebar -->
    <nav id="sidebar">
      <div id="dismiss">
        <i class="fas fa-arrow-left"></i>
      </div>
      <div class="sidebar-header" style="text-align: center;">
        <h4>SMART DOOR LOCK</h4>
      </div>
      <ul id="nav-list">
        <li class="active"><a class="nav-item" href="#">Dashboard</a></li>
        <li><a class="nav-item" href="{% url 'logs' %}">Logs</a></li>
        <li><a class="nav-item" href="{% url 'form' %}">Add Member</a></li>
        <li><a class="nav-item" href="{% url 'verify_otp' %}">Unlock Access</a></li>
        <li><a class="nav-item" href="{% url 'bot_response' %}">Chatbot Simulation</a></li>
      </ul>
      <ul class="list-unstyled logout">
        <li><a href="{% url 'logout' %}" id="logout">Logout</a></li>
      </ul>
    </nav>
    <div id="content">
      <nav class="navbar navbar-expand-lg navbar-light bg-light">
        <div class="container-fluid">
          <button type="button" id="sidebarCollapse" class="btn btn-info" style="background-color: #4e73df;">
            <i class="fas fa-align-justify"></i>
          </button>
          <button id="btn1" class="btn d-inline-block d-lg-none ml-auto" type="button" data-
toggle="collapse" data-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-
expanded="false">{{request.user}}</button>
          <div class="collapse navbar-collapse">
            <ul class="nav navbar-nav ml-auto">
              <li class="nav-item active"><a class="nav-link" href="#">admin</a></li>
            </ul>
          </div>
          <div class="dropdown-menu shadow dropdown-menu-right animated--grow-
in" id="navbarSupportedContent">
            <a class="dropdown-item" href="#"><i class="fas fa-user fa-sm fa-fw mr-2 text-gray-
400"></i> Profile</a>
            <a class="dropdown-item" href="#"><i class="fas fa-sign-out-alt fa-sm fa-fw mr-2 text-gray-
400"></i> Logout</a>
          </div>
        </div>
      </nav>
    </div>
    <div class="overlay"></div>
  </body>
```

JavaScript Code:

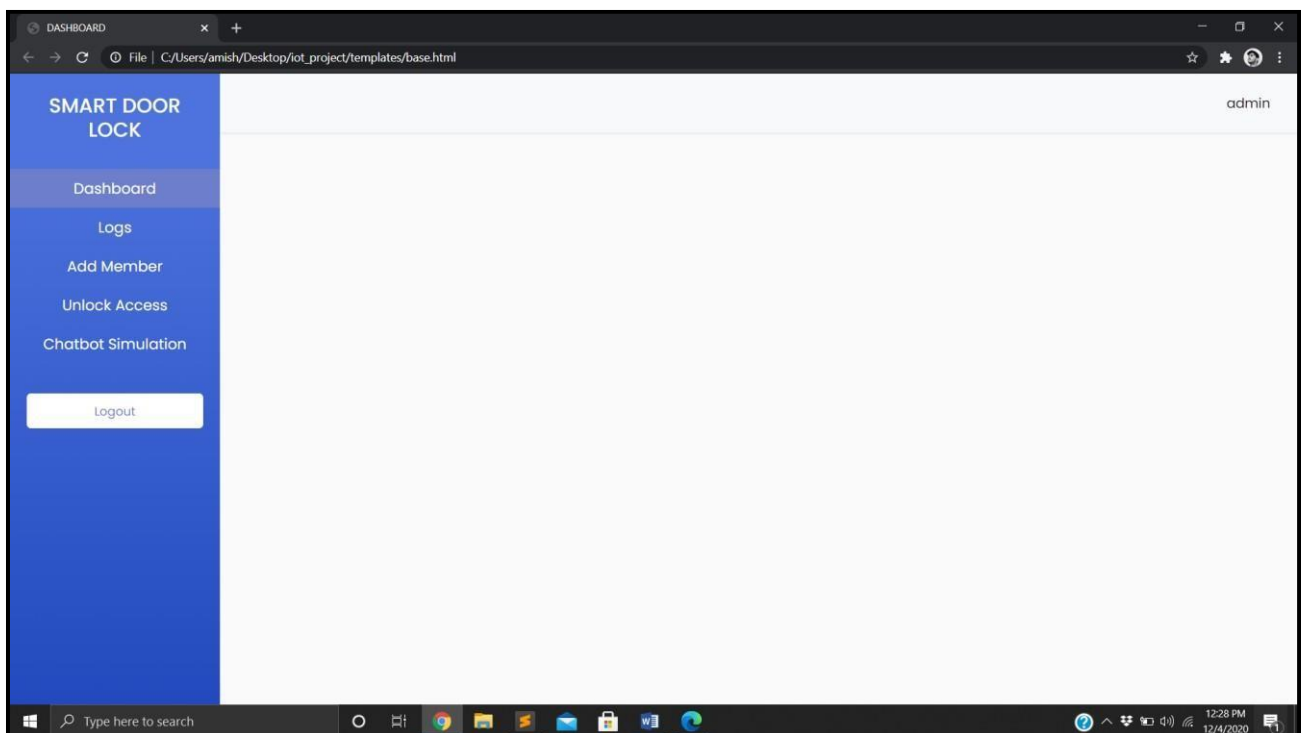
```
$(document).ready(function () {
    $('#sidebarCollapse').on('click', function () {
        $('#sidebar').toggleClass('active');
    });
});

$(document).ready(function () {
    $("#sidebar").mCustomScrollbar({
        theme: "minimal"
    });
});

$('#dismiss, .overlay').on('click', function () {
    $('#sidebar').removeClass('active');
    $('.overlay').removeClass('active');
});

$('#sidebarCollapse').on('click', function () {
    $('#sidebar').addClass('active');
    $('.overlay').addClass('active');
    $('.collapse.in').toggleClass('in');
    $('a[aria-expanded=true]').attr('aria-
expanded', 'false');
});
});
```

OUTPUT:



4.2 DJANGO – SQLite Database

OUTPUT:

Framework Code:

```
@login_required(login_url='login')
@admin_only
def registerPage(request):
    if request.method == 'POST':
        user_reg = User(first_name=request.POST.get('first_name'),
                        password=request.POST.get('password1'),
                        username=request.POST.get('username'))

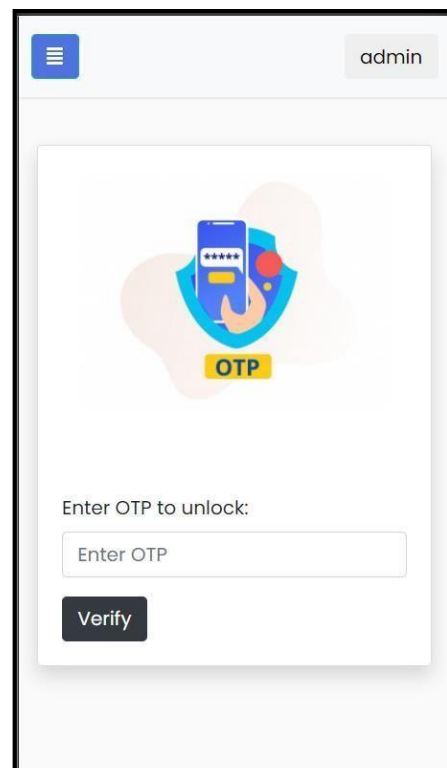
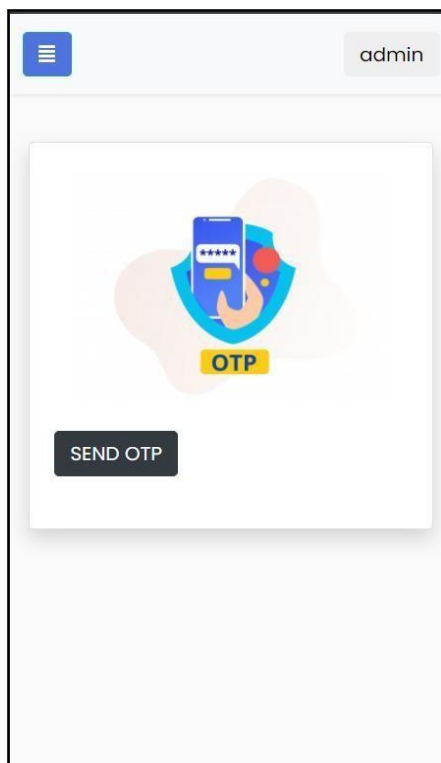
        try:
            user_reg.save()
            groups = Group.objects.get(name=request.POST.get('access_level'))
            groups.user_set.add(user_reg)
            context = {'folder_name': request.POST.get('first_name')}
            return render(request, "dataset.html", context)
        except IntegrityError:
            messages.info(request, "Username already present!!")
    return render(request, 'form.html')
```

REGISTRATION FORM CODE:

```
<form id="registration" method="POST">
  {% csrf_token %}
  <div class="form-row">
    <div class="col">
      <div class="form-group">
        <label for="name">Full Name</label>
        <input id="name" class="form-
control" type="text" name="first_name" placeholder=" Enter Name" required>
      </div>
      <div class="form-group">
        <label for="username">Username</label>
        <input class="form-control" type="text" id="pswd" name="username" required>
      </div>
      <div class="form-group">
        <label for="pswd">Password</label>
        <input class="form-control" type="password" id="pswd" name="password1" required>
      </div>
      <div class="form-group">
        <label for="access_level">Access Level</label>
        <select class="form-control" id="access_level" form="registration" name="access_level" required>
          <option>ADMIN</option>
          <option>VIEW_UNLOCK</option>
          <option>VIEW</option>
        </select>
      </div>
    </div>
  </div>
  <br>
  <button type="submit" onclick="myFunction()" class="btn btn-primary">Register</button>
</form>
```

AJAX SCRIPT:

```
$(function(){  
    $('#sendbtn').click(function(){  
        $.ajax({  
            url: 'send_otp',  
            type: 'GET',  
            success:function(response){  
                console.log(response);  
                alert("OTP sent successfully!")  
            },  
            error: function(error){  
                console.log(error);  
                alert("Fail to send OTP");  
            }  
        });  
    });  
});
```



LOGS PAGE (DATA FROM SQLITE DATABASE)

The screenshot shows the 'STATS' page of the IoT Project application. The sidebar on the left contains the following menu items: Dashboard, Logs, Add Member, Unlock Access, Chatbot Simulation, and a Logout button. The main content area is titled 'STATS' and features a 'LOCK STATUS' section with a blue 'UNLOCK' button and a lock icon. Below this is a 'LOGS TABLE' with a search bar and a table of logs.

ID	Person	TIME
7807	kshiti	Dec. 3, 2020, 12:13 a.m.
7806	abhijeet	Dec. 3, 2020, 12:10 a.m.
7805	Amish	Dec. 3, 2020, 12:09 a.m.
7804	Amish	Dec. 3, 2020, 12:09 a.m.
7803	Amish	Dec. 3, 2020, 12:09 a.m.
7802	Amish	Dec. 3, 2020, 12:09 a.m.

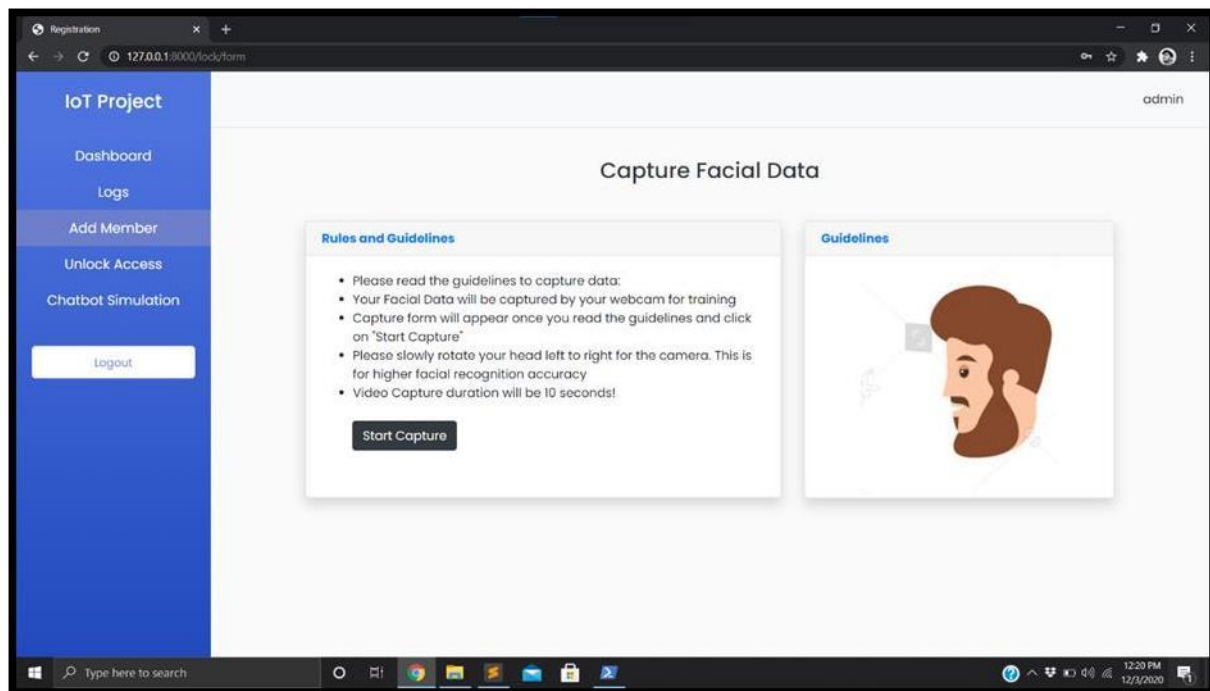
MEMBER REGISTRATION FORM

STEP 1: REGISTERING USER AND SETTING HIS/HER ACCESS LEVEL

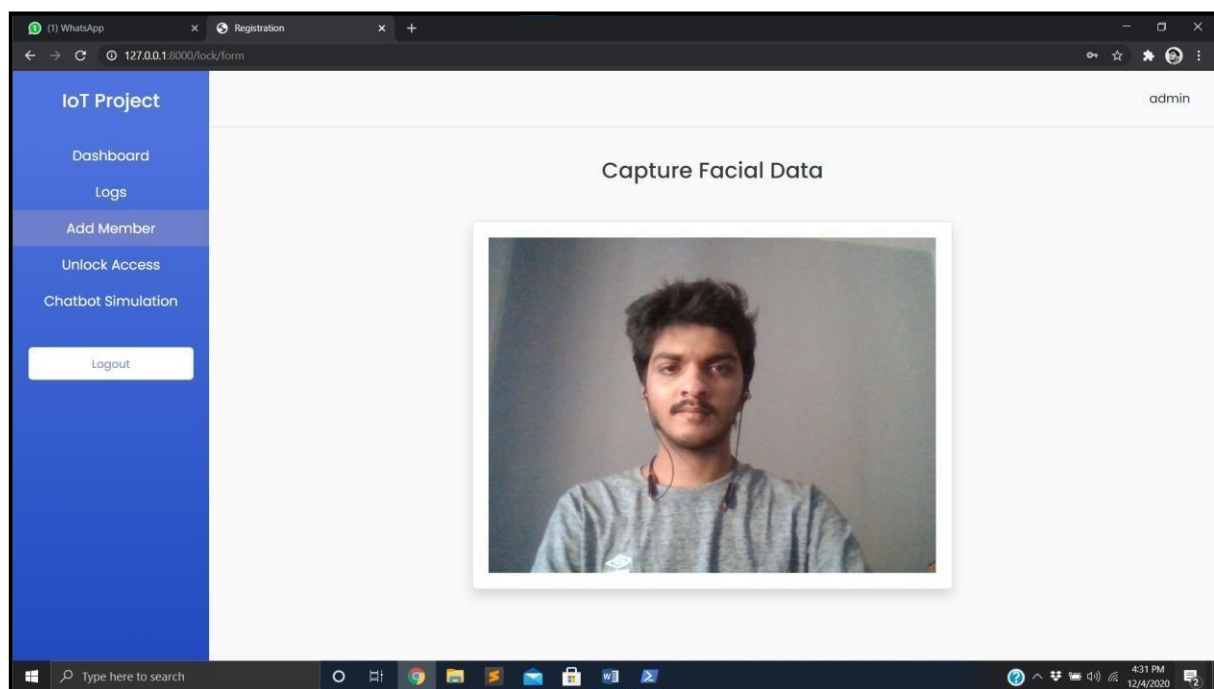
The screenshot shows the 'Add Member' registration form in the IoT Project application. The sidebar on the left contains the following menu items: Dashboard, Logs, Add Member, Unlock Access, Chatbot Simulation, and a Logout button. The main content area is titled 'Add Member' and contains a form with the following fields:

- Full Name: Abhijeet
- Username: heisenberg
- Password: (masked with dots)
- Access Level: (dropdown menu with options: ADMIN, ADMIN, VIEW_UNLOCK, VIEW)

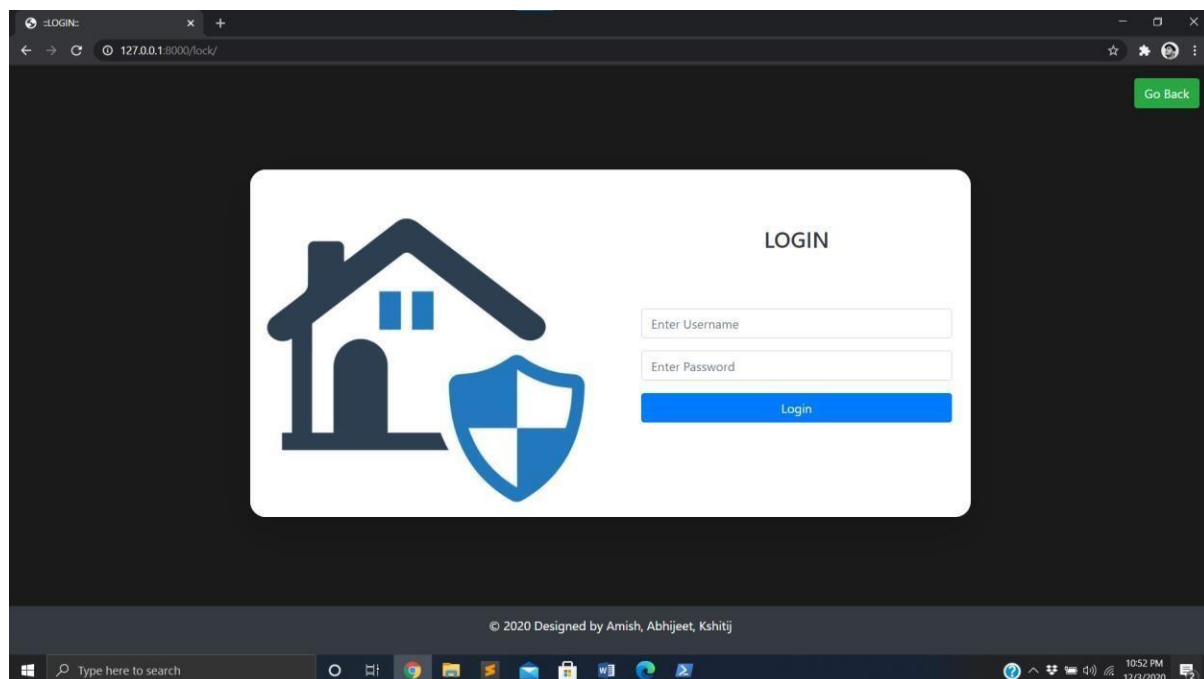
STEP 2: CAPTURE FACIAL DATA GUIDELINES



STEP 3: WEBCAM IS ACCESSED BY THE WEBSITE TO CAPTURE THE DATA FOR TRAINING FACIAL RECOGNIZER.



LOGIN PAGE



CHATBOT UI
(Simulation of the Voice Assistant)

Experiment Number: 8

Work Breakdown Structure

Title: To prepare the WBS for the case study <case study name>

Pre requisites:

Learning Objectives (Bloom's Taxonomy): Evaluating

Theory:

1. Work Breakdown Structure(WBS)

WBS provides a hierarchical structure that outlines the activities or work that needs to be done in order to complete the project scope. The WBS also provides a bridge or link between the project's scope and the detailed project plan that will be entered into a project management software package. The WBS represents a logical decomposition of the work to be performed and focuses on how the product, service, or result is naturally subdivided. It is an outline of what work is to be performed. The Wbs also provides a framework for developing a tactical plan to structure the project work. The total scope of the project is divided into specific deliverables that can be more easily managed. In short, the WBS provides an outline for all of the work the project team will perform.

The WBS decomposes, or subdivides the project into smaller components and more manageable units of work called work packages. Work packages provides a logical basis for defining the project activities and assigning resources to those activities so that all the project work is identified. A work package may be viewed as a hierarchy that starts with the project itself. The project is then decomposed into phases, with each phase having one or more deliverables as defined in the deliverable definition table and deliverable structure chart. More specifically, each phase should provide at least one specific deliverable—that is, a tangible and verifiable piece of work. Subsequently, activities or tasks are identified in order to produce the project's deliverables.

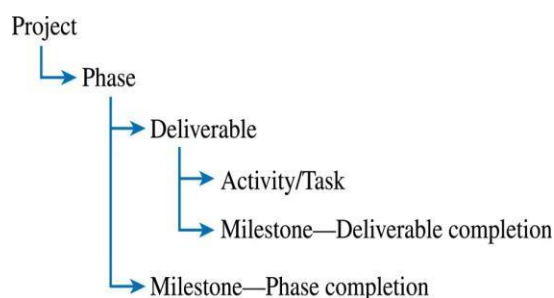


Fig.1. Work Package

A WBS can be represented by two different formats namely the outline format and graphical format.

Outline format(indented) -resembles an outline with major tasks at first level and subtask listed under each in hierarchical fashion

Graphical format (organizational)-hierarchical block diagram with major task blocks at top level and subtask blocks for each connected to them shown in Fig.2.

The WBS shown in Fig.3 follows an outline format with a commonly used decimal numbering system that allows for continuing levels of detail.

WBS Chart Example

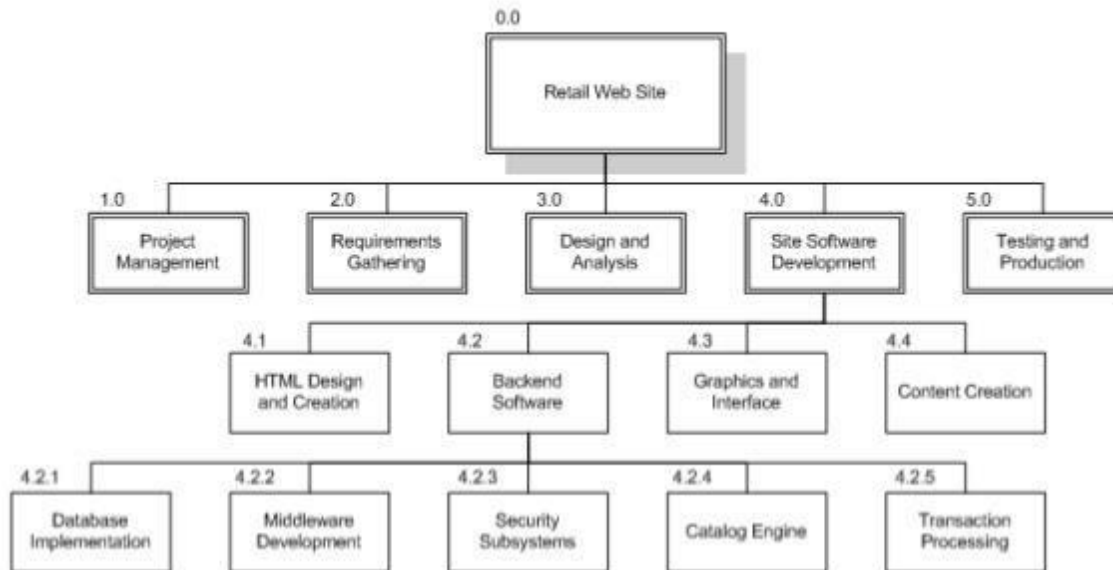


Fig.2 WBS-Graphical Format

WBS Outline Example

- 0.0 Retail Web Site
 - 1.0 Project Management
 - 2.0 Requirements Gathering
 - 3.0 Analysis & Design
 - 4.0 Site Software Development
 - 4.1 HTML Design and Creation
 - 4.2 Backend Software
 - 4.2.1 Database Implementation
 - 4.2.2 Middleware Development
 - 4.2.3 Security Subsystems
 - 4.2.4 Catalog Engine
 - 4.2.5 Transaction Processing
 - 4.3 Graphics and Interface
 - 4.4 Content Creation
 - 5.0 Testing and Production

Fig.3.WBS-Outline Format

Hardware & Software requirements:

Hardware: Computer lab, Internet connection for literature survey

Software: Libre Office for preparing document

Procedure:**Procedure for creating WBS**

1. Create the work Package for your case study
2. Develop the WBS according to the work package in any format(either Graphical or Outline)



Level	WBS Code	Element Name
1	1	Smart Door Lock System
2	1.1	Image Preprocessing
3	1.1.1	Gray Scale Processing
3	1.1.2	Noise Removal
3	1.1.3	Thinning and Binarization
2	1.2	Segmentation
3	1.2.1	Divide Text into Rows
3	1.2.2	Divide Rows into Words
3	1.2.3	Divide Words into Letters
2	1.3	Feature Extraction
3	1.3.1	Edge Detection
3	1.3.2	Generation of Binary Glyphs
2	1.4	Remote Lock System
3	1.4.1	Log Record
3	1.4.2	Send Alert
3	1.4.3	Unlock Door OTP
2	1.5	Recognition using Neural Network
3	1.5.1	Training on Dataset
3	1.5.2	Facial Recognition
3	1.5.3	Log Record

Conclusion:

References:

1. "Information Technology Project Management", Jack T. Marchewka, 4th edition, Wiley India, 2014.
2. <http://www.projectmanagementdocs.com/project-planning-templates>

Experiment Number: 9

Schedule Management Plan

Title: To prepare the Schedule Management Plan with network diagrams and timeline charts.

Pre requisites: Function Point Estimation and COCOMO techniques

Learning Objectives (Bloom's Taxonomy): Create

Theory:

Project Scheduling

Using FP estimation, the size of the software product is estimated first. Then COCOMO estimation is used to estimate schedule S (in months) as a function of effort in PMs. Effort distribution and schedule distribution in phases are different. A recommended distribution of effort across the definition and development phases is often referred to as the 40–20–40 rule.

1. Forty percent of all effort is allocated to front-end analysis and design.
2. A similar percentage is applied to back-end testing.
3. You can correctly infer that coding (20 percent of effort) is de-emphasized.

This effort distribution should be used as a guideline only. With effort and overall schedule decided, average project resources are fixed. With manpower ramp-up and effort distribution, milestones can be decided. Fig. 1 gives a breakup of the above distribution.

When creating a software project schedule, the planner begins with a set of tasks (the work breakdown structure). Effort, duration, and start date are then input for each task. In addition, tasks may be assigned to specific individuals. Also, the sequencing is to be determined using the interdependencies of tasks and resource sharing to calculate the total time of the project. For this, network diagrams are helpful, such as PERT, CPM, AON, CCM etc.

If automated tools are used, the work breakdown is input as a task network or task outline. As a consequence of this input, a timeline chart, also called a Gantt chart, is generated for the entire project. Fig.1 illustrates the format of a timeline chart.

All project tasks (for concept scoping) are listed in the left-hand column. The horizontal bars indicate the duration of each task. When multiple bars occur at the same time on the calendar, task concurrency is implied. The diamonds indicate milestones.

Once the information necessary for the generation of a timeline chart has been input, the majority of software project scheduling tools produce project tables—a tabular listing of all project tasks, their planned and actual start- and end-dates, and a variety of related information. Used in conjunction with the timeline chart, project tables enable the project manager to track progress. Prepare the table as shown in Fig. 2

Hardware & Software requirements:

Hardware: Computer system in a lab, Internet connection for literature survey

Software: Microsoft Office or Libre Office for preparing document

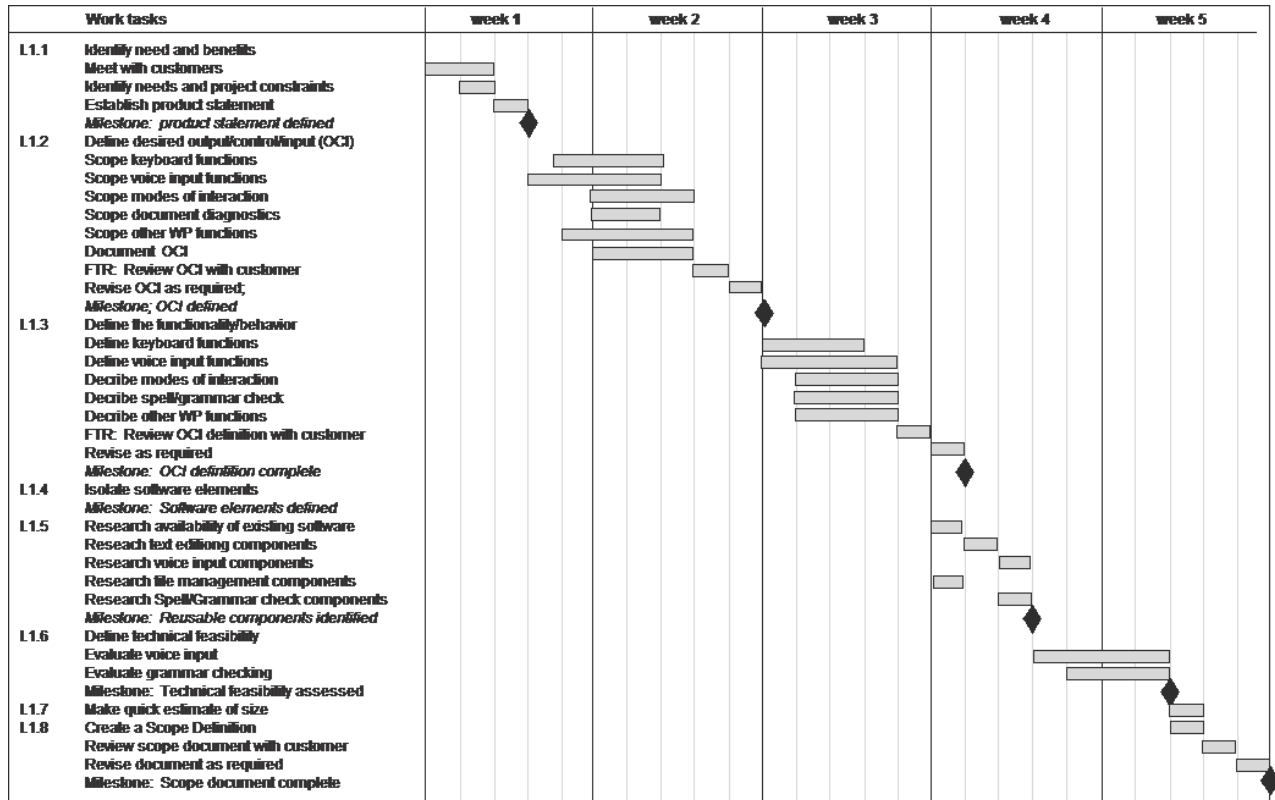
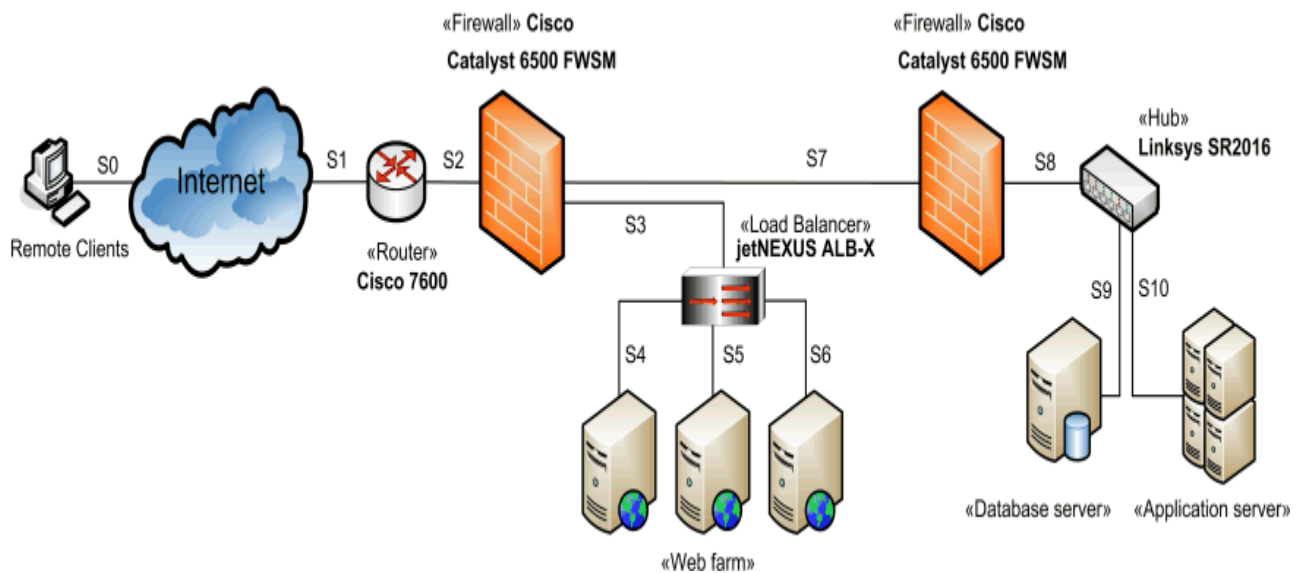


Fig.1. Timeline chart

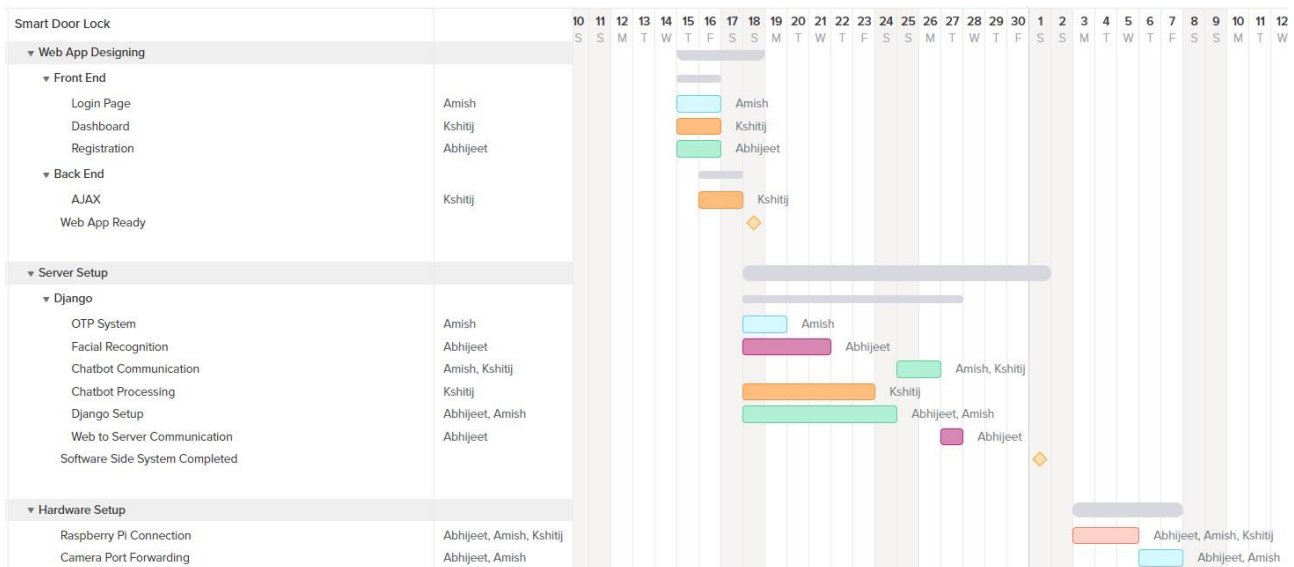
Work tasks	Planned start	Actual start	Planned complete	Actual complete	Assigned person	Effort allocated	Notes
1.1.1 Identify needs and benefits Meet with customers Identify needs and project constraints Establish product statement <i>Milestone: Product statement defined</i>	wk1, d1 wk1, d2 wk1, d3 wk1, d3	wk1, d1 wk1, d2 wk1, d3 wk1, d3	wk1, d2 wk1, d2 wk1, d3 wk1, d3	wk1, d2 wk1, d2 wk1, d3 wk1, d3	BLS JPP BLS/JPP	2 p-d 1 p-d 1 p-d	Scoping will require more effort/time
1.1.2 Define desired output/control/input (OCI) Scope keyboard functions Scope voice input functions Scope modes of interaction Scope document diagnostics Scope other WP functions Document OCI FTR: Review OCI with customer Revise OCI as required <i>Milestone: OCI defined</i>	wk1, d4 wk1, d3 wk2, d1 wk2, d1 wk1, d4 wk2, d1 wk2, d3 wk2, d4 wk2, d5	wk1, d4 wk1, d3 wk1, d4 	wk2, d2 wk2, d2 wk2, d3 wk2, d2 wk2, d3 wk2, d3 wk2, d3 wk2, d4 wk2, d5	 	BLS JPP MLL BLS JPP MLL all all	1.5 p-d 2 p-d 1 p-d 1.5 p-d 2 p-d 3 p-d 3 p-d 3 p-d	
1.1.3 Define the function/behavior							

Procedure:

1. Draw the network diagram to distribute the time estimated among the tasks, depict their dependencies, determine the critical path and hence the project duration.



- Using the duration obtained, decide the start and end dates and prepare the Gantt chart using any open source tool.



Conclusion: The Schedule Management Plan was developed for Smart Door Lock System

References:

- Software Engineering: A Practitioner's Approach, 6th edition, by Roger S. Pressman
- <http://www.projectmanagementdocs.com/project-planning-templates>

<<include other references>>